HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



ASSIGNMENT 01 – INTELLIGENT SYSTEM DEVELOPMENT

Học phần: Phát triển các hệ thống thông minh

Giảng viên hướng dẫn: Thầy Trần Đình Quế

Sinh viên thực hiện: Phạm Mai Linh

Mã sinh viên: B23DCCN488

Hà Nội – 2026

MỤC LỤC

PHẦN I – LÝ THUYẾT 4
1. Từ Intelligence đến Intelligent System 4
2. Hai bài toán và phát biểu bài toán chính thức 4
2.1. Hệ thống chẩn đoán nguy cơ tiểu đường 4
2.2. Hệ thống dự đoán giá nhà 5
3. Data Representation – Biểu diễn dữ liệu 5
4. Supervised Learning 6
5. Baseline 6
6. Các mô hình Machine Learning 6
7. Độ đo đánh giá 7
8. Controlled Experiments 7
9. Từ Model đến Application 8
PHẦN II – HỆ THỐNG 1: CHẨN ĐOÁN NGUY CƠ TIỂU ĐƯỜNG 8
10. Mục tiêu hệ thống 8
11. Exploratory Data Analysis – Các biểu đồ và giải thích 10
Biểu đồ 1 – Phân bố Outcome 11
Biểu đồ 2 – Glucose theo Outcome 12
Biểu đồ 3 – BMI theo Outcome 13
Biểu đồ 4 – Ma trận tương quan 14
Biểu đồ 5 – Toàn bộ feature 16
12. Data Representation và preprocessing 17
13. Baseline và 5 mô hình 17
14. Experiment 1 – Model Comparison 23
15. Experiment 2 – Hyperparameter Investigation 23
16. Experiment 3 – Representation Investigation 24
17. Final Model và lưu model 24
PHẦN III – HỆ THỐNG 2: DỰ ĐOÁN GIÁ NHÀ 25
18. Mục tiêu 25
19. EDA – Biểu đồ giá nhà và giải thích 26

Biểu đồ 1 – Phân phối Price....26
Biểu đồ 2 – Area và Price....27
Biểu đồ 3 – Price theo Bedrooms....28
Biểu đồ 4 – Correlation của biến số....29
20. Train/Test và Representation cho giá nhà....31
21. Experiment 1 – Model Comparison....35
22. Experiment 2 – Hyperparameter Investigation....35
23. Experiment 3 – Representation / Feature Investigation....36
24. Final Model – Giá nhà....37
PHẦN IV – TRIỂN KHAI WEB....37
25. Kiến trúc Web....37
26. Kiến trúc Mobile....39

PHẦN I – LÝ THUYẾT

1. Từ Intelligence đến Intelligent System

Theo Assignment 01, trí thông minh trong bài được hiểu theo nghĩa vận hành: hệ thống có khả năng **tiếp nhận thông tin, biểu diễn thông tin dưới dạng tính toán, học mối quan hệ từ các ví dụ, dự đoán cho trường hợp mới và hỗ trợ một quyết định/ứng dụng.**

Vì vậy, một mô hình Machine Learning đơn lẻ chưa phải là toàn bộ hệ thống thông minh. Hệ thống hoàn chỉnh phải có chuỗi:

Môi trường/Dữ liệu → Input → Representation → Learning/Model → Decision → Output → Application

Trong bài này:

- **Hệ tiểu đường:** thông tin sức khỏe → vector đặc trưng → mô hình phân loại → dự đoán có/không có nguy cơ tiểu đường.
- **Hệ giá nhà:** thông tin bất động sản → vector đặc trưng → mô hình hồi quy → giá nhà dự đoán.

Assignment nhấn mạnh rằng mô hình không trực tiếp nhận "thế giới thực", mà nhận một **biểu diễn tính toán** của những thông tin được lựa chọn.

2. Hai bài toán và phát biểu bài toán chính thức

2.1. Hệ thống chẩn đoán nguy cơ tiểu đường

Bài toán thực tế: hỗ trợ sàng lọc nguy cơ tiểu đường dựa trên các chỉ số sức khỏe.

Input: 8 đặc trưng: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age.

Target: Outcome, trong đó:

- 0: không có tiểu đường theo nhãn dữ liệu.
- 1: có tiểu đường theo nhãn dữ liệu.

Loại bài toán: Classification nhị phân.

Phát biểu chính thức:

Với một vector đặc trưng sức khỏe chưa từng xuất hiện trong tập kiểm tra, dự đoán lớp Outcome $\in \{0,1\}$.

2.2. Hệ thống dự đoán giá nhà

Bài toán thực tế: hỗ trợ ước lượng giá nhà dựa trên các thuộc tính của bất động sản.

Input: các đặc trưng như diện tích, mặt tiền, đường vào, hướng nhà, hướng ban công, số tầng, số phòng ngủ, số phòng tắm, pháp lý và tình trạng nội thất.

Target: Price.

Loại bài toán: Regression.

Phát biểu chính thức:

Với một vector đặc trưng bất động sản chưa từng xuất hiện trong tập kiểm tra, dự đoán giá nhà **Price** dưới dạng một giá trị số.

3. Data Representation – Biểu diễn dữ liệu

Với dữ liệu có cấu trúc, mỗi quan sát được biểu diễn bởi một vector:

$$x_i = [x_{i1}, x_{i2}, ..., x_{id}] \in R^d$$

Toàn bộ tập dữ liệu được biểu diễn bởi:

$$D = \{(x_i, y_i)\}; i = 1..N$$

Trong đó:

- x_i: biểu diễn đầu vào của quan sát thứ i.
- d: số đặc trưng sau khi biểu diễn.
- y_i: target.
- N: số quan sát.
- X: ma trận đặc trưng.
- y: vector mục tiêu.

Điểm quan trọng: **raw feature ≠ encoded feature ≠ model input** nếu quá trình tiền xử lý thay đổi biểu diễn.

Tiểu đường

Các biến đều là số. Một số giá trị **0** ở **Glucose, BloodPressure, SkinThickness, Insulin, BMI** có thể không mang ý nghĩa sinh lý hợp lý; trong notebook, các giá trị này được xem là thiếu và thay bằng median của tập train thông qua pipeline.

Giá nhà

Dữ liệu có cả biến số và biến phân loại. Các biến phân loại được mã hóa bằng **One-Hot Encoding**; biến số được điền missing bằng median. Vì các mô hình và preprocessing nằm trong Pipeline, cùng một biểu diễn được áp dụng cho train, test và dữ liệu mới.

4. Supervised Learning

Mô hình học một hàm:

$$\hat{y} = f_\theta(x)$$

Mục tiêu là tìm tham số θ^* giúp sai số dự đoán nhỏ trên dữ liệu học, đồng thời có khả năng **generalization** – dự đoán tốt cho dữ liệu chưa từng thấy.

Train/Test

Dữ liệu được chia thành:

- **Training set:** dùng để học.
- **Test set:** chỉ dùng để đánh giá cuối cùng.

Trong bài, sử dụng tỷ lệ 80/20 và random_state = 42. Với classification sử dụng stratify = y.

Không sử dụng test set để lựa chọn mô hình hoặc tinh chỉnh tham số, nhằm tránh data leakage.

5. Baseline

Baseline là mốc tham chiếu đơn giản nhất.

- Classification: DummyClassifier(strategy="most_frequent").
- Regression: DummyRegressor(strategy="mean").

Baseline trả lời câu hỏi: **"Các mô hình Machine Learning có thực sự tốt hơn một chiến lược đơn giản hay không?"**

Assignment yêu cầu phải có baseline trước khi đánh giá các mô hình phức tạp hơn.

6. Các mô hình Machine Learning

Classification – Tiểu đường

1. Logistic Regression – học ranh giới tuyến tính và xác suất lớp.
2. K-Nearest Neighbors – dự đoán dựa trên các điểm dữ liệu gần nhất.
3. Decision Tree – chia không gian đặc trưng thành các vùng bằng các luật rẽ nhánh.
4. Random Forest – kết hợp nhiều cây quyết định để tăng tính ổn định.

5. Support Vector Machine – tìm biên phân tách với margin phù hợp.

Regression – Giá nhà

1. Linear Regression – mô hình hóa quan hệ tuyến tính.
2. Ridge Regression – Linear Regression có regularization L2.
3. Decision Tree Regressor – phân vùng không gian đặc trưng và dự đoán theo vùng.
4. Random Forest Regressor – trung bình nhiều cây hồi quy.
5. Gradient Boosting Regressor – xây dựng tuần tự các mô hình để sửa sai số của mô hình trước.

Với mỗi mô hình, bài cần quan tâm: mô hình nhận representation nào, học quan hệ gì, cấu trúc/tham số nào được học, tiêu chí học, giả định, ưu điểm và hạn chế.

7. Độ đo đánh giá

Classification

- **Accuracy:** tỷ lệ dự đoán đúng trên toàn bộ mẫu.
- **Precision:** trong các mẫu được dự đoán dương tính, bao nhiêu mẫu thực sự dương tính.
- **Recall:** trong các mẫu thực sự dương tính, mô hình phát hiện được bao nhiêu.
- **F1-score:** trung bình điều hòa giữa Precision và Recall.
- **Confusion Matrix:** cho biết TP, TN, FP, FN.

Trong hệ thống sàng lọc bệnh, Recall có ý nghĩa đặc biệt vì bỏ sót một trường hợp có nguy cơ có thể nghiêm trọng. Tuy nhiên, không thể chỉ tối ưu Recall mà bỏ qua Precision và F1.

Regression

- **MAE:** sai số tuyệt đối trung bình, dễ diễn giải theo đơn vị của giá.
- **MSE:** bình phương sai số trung bình, phạt mạnh sai số lớn.
- **RMSE:** căn bậc hai của MSE, cùng đơn vị với target.
- **R^2:** mức độ giải thích biến thiên của target; càng gần 1 càng tốt.

Không sử dụng Accuracy cho bài toán giá nhà vì đây là regression.

8. Controlled Experiments

Assignment yêu cầu tối thiểu 3 thí nghiệm có câu hỏi rõ ràng.

Experiment 1 – Model Comparison

Câu hỏi: Với cùng một train/test split và cùng protocol đánh giá, mô hình nào hoạt động tốt nhất?

Experiment 2 – Hyperparameter Investigation

Câu hỏi: Thay đổi một hyperparameter có ảnh hưởng đến kết quả không?

- Classification: khảo sát k của KNN.
- Regression: khảo sát n_estimators của Random Forest.

Experiment 3 – Representation / Feature Investigation

Câu hỏi: Thay đổi representation có ảnh hưởng đến khả năng học không?

- Classification: so sánh dữ liệu có chuẩn hóa và không chuẩn hóa.
- Regression: so sánh nhóm feature số với nhóm feature số + categorical.

Mỗi experiment phải thay đổi có kiểm soát, không thay đổi hàng loạt tham số cùng lúc.

9. Từ Model đến Application

Một model đã train chưa phải hệ thống hoàn chỉnh. Application phải thực hiện:

Input → Feature Representation → Preprocessing → Model → Prediction → Output

Web và Mobile trong bài sử dụng **cùng final model và cùng logic preprocessing**, thông qua REST API Flask. Mobile gửi JSON đến API, API trả về prediction và probability/giá dự đoán.

Điều này đảm bảo dữ liệu mới được biểu diễn giống dữ liệu lúc training.

PHẦN II – HỆ THỐNG 1: CHẨN ĐOÁN NGUY CƠ TIỂU ĐƯỜNG

10. Mục tiêu hệ thống

Hệ thống nhận 8 thông tin đầu vào liên quan đến sức khỏe và dự đoán nhãn Outcome.

Đây là **hệ thống hỗ trợ sàng lọc học thuật**, không phải công cụ chẩn đoán y khoa. Kết quả chỉ có ý nghĩa trong phạm vi dataset và mô hình đã học.

Đọc dataset:

```
# TÌM DATASET TIỂU ĐƯỜNG

DIABETES_PATH = Path(r"C:\DATA\diabetes")

def resolve_csv(path, fallback=None):
    p = Path(path)
    if p.is_file():
        return p
    if p.is_dir():
        files = sorted(p.glob("*.csv"))
        if files:
            return files[0]
    if fallback:
        fp = Path(fallback)
        if fp.is_file():
            return fp
    # Tìm một số vị trí phổ biến để notebook có thể kiểm tra độc lập
    candidates = sorted(Path(".").glob("**/*.csv"))
    for f in candidates:
        if "diabetes" in f.name.lower():
            return f
    raise FileNotFoundError(
        f"Không tìm thấy dataset tiểu đường. Hãy đặt file tại {path} "
        "hoặc kiểm tra lại C:\DATA\diabetes."
    )

DIABETES_CSV = resolve_csv(DIABETES_PATH, "/mnt/data/diabetes(5).csv")
diabetes = pd.read_csv(DIABETES_CSV)

print("Đường dẫn dữ liệu:", DIABETES_CSV)
print("Kích thước:", diabetes.shape)
display(diabetes.head())
```

```
Đường dẫn dữ liệu: data_diabetes.csv
Kích thước: (768, 9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Giải thích dữ liệu

Dataset có 768 quan sát và 8 biến đầu vào cùng target Outcome. Tất cả feature hiện đang ở dạng số. Outcome là biến phân loại nhị phân.

Các biến sức khỏe như Glucose và BMI có thể mang thông tin phân biệt hai nhóm. Tuy nhiên, mối quan hệ quan sát được chỉ là quan hệ trong dataset, không được diễn giải thành quan hệ nhân quả y khoa.

```python
# PHÂN TÍCH FEATURE VÀ TARGET
feature_cols_d = [c for c in diabetes.columns if c != "Outcome"]
target_d = "Outcome"

display(pd.DataFrame({
    "Feature": feature_cols_d,
    "Type": [str(diabetes[c].dtype) for c in feature_cols_d],
    "Meaning": [
        "Số lần mang thai",
        "Nồng độ glucose",
        "Huyết áp tâm trương",
        "Độ dày da",
        "Insulin",
        "Chỉ số BMI",
        "Hệ số phả hệ tiểu đường",
        "Tuổi"
    ]
}))
print("Target:", target_d)
print("Phân bố target:")
display(diabetes[target_d].value_counts().rename_axis("Outcome").to_frame("Count"))
```

	Feature	Type	Meaning
0	Pregnancies	int64	Số lần mang thai
1	Glucose	int64	Nồng độ glucose
2	BloodPressure	int64	Huyết áp tâm trương
3	SkinThickness	int64	Độ dày da
4	Insulin	int64	Insulin
5	BMI	float64	Chỉ số BMI
6	DiabetesPedigreeFunction	float64	Hệ số phả hệ tiểu đường
7	Age	int64	Tuổi

```
Target: Outcome
Phân bố target:
```

Outcome	Count
0	500
1	268

11. Exploratory Data Analysis – Các biểu đồ và giải thích

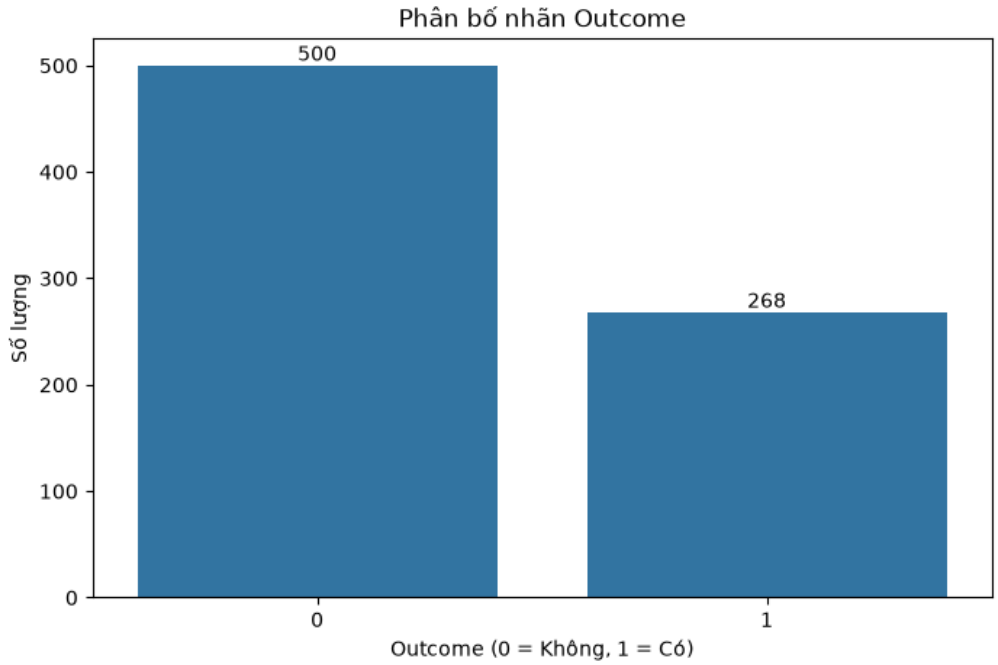
Các biểu đồ không chỉ nhằm minh họa mà giúp trả lời các câu hỏi:

- Dữ liệu có mất cân bằng không?
- Feature nào có phân phối khác nhau giữa hai lớp?
- Có outlier hoặc giá trị bất thường không?
- Các feature có tương quan với nhau không?

Biểu đồ 1 – Phân bố Outcome

Nếu hai lớp chênh lệch mạnh, Accuracy có thể gây hiểu nhầm; vì vậy cần xem Precision, Recall, F1 và Confusion Matrix.

```
# BIỂU ĐỒ 1
plt.figure(figsize=(8,5))
ax=sns.countplot(data=diabetes, x="Outcome")
plt.title("Phân bố nhãn Outcome")
plt.xlabel("Outcome (0 = Không, 1 = Có)")
plt.ylabel("Số lượng")
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}", (p.get_x()+p.get_width()/2,p.get_height()),
                ha="center", va="bottom")
plt.show()
```



Cách đọc Biểu đồ 1 – Phân bố Outcome

Bước 1 – Đọc trục: Trục X là Outcome (0 hoặc 1). Trục Y là **số lượng bệnh nhân/mẫu** thuộc mỗi lớp. Đây là countplot nên chiều cao cột chính là số quan sát, không phải phần trăm.

Bước 2 – So sánh hai cột: Cột nào cao hơn nghĩa là lớp đó có nhiều mẫu hơn. Nếu một cột cao hơn rất nhiều, dữ liệu có dấu hiệu **mất cân bằng lớp**.

Bước 3 – Hiểu ý nghĩa: Nếu lớp 0 chiếm đa số, một mô hình có thể đạt Accuracy khá cao chỉ bằng cách thiên về lớp 0. Vì vậy Accuracy một mình chưa đủ để đánh giá hệ thống chẩn đoán.

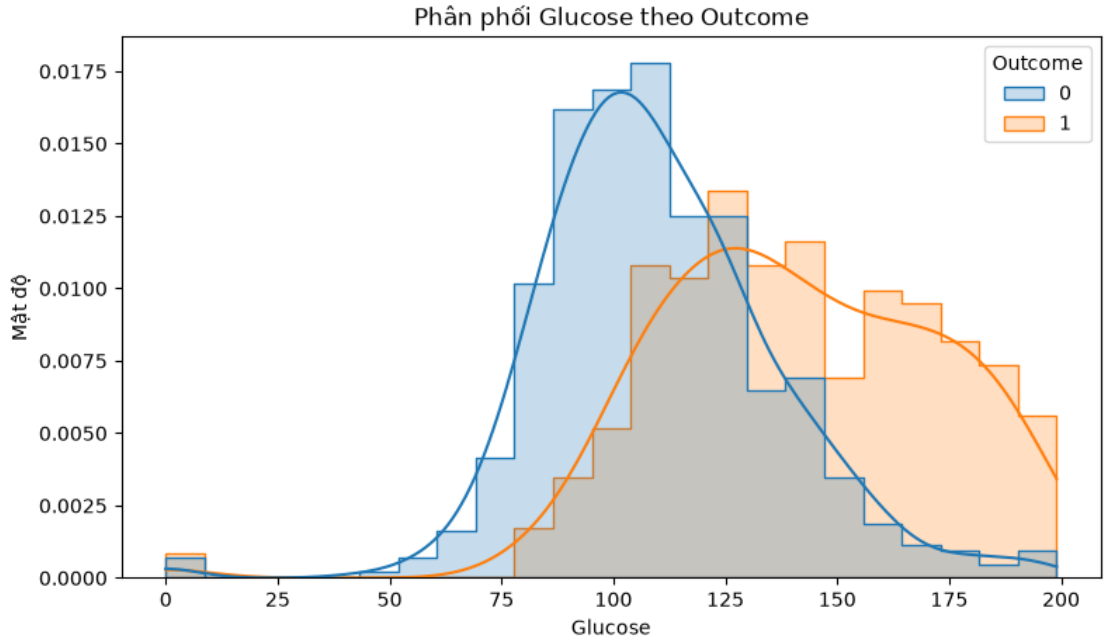
Bước 4 – Liên hệ mô hình: Cần xem thêm Precision, Recall, F1 và Confusion Matrix. Đặc biệt, Recall của lớp dương tính giúp trả lời: "Trong những người thực sự thuộc lớp 1, hệ thống phát hiện được bao nhiêu người?"

Câu nói khi thuyết trình: "Em đọc biểu đồ này trước tiên để kiểm tra phân bố nhãn. Nếu hai lớp không cân bằng, em không thể chỉ nhìn Accuracy; em phải xem thêm Recall, Precision, F1 và ma trận nhầm lẫn."

Biểu đồ 2 – Glucose theo Outcome

Nếu phân phối Glucose của hai nhóm lệch nhau, Glucose có khả năng cung cấp thông tin hữu ích cho mô hình. Tuy nhiên, biểu đồ này chỉ cho thấy sự khác biệt phân phối, không chứng minh nguyên nhân.

```
# BIỂU ĐỒ 2
plt.figure(figsize=(9,5))
sns.histplot(data=diabetes, x="Glucose", hue="Outcome", kde=True, stat="density",
             common_norm=False, element="step")
plt.title("Phân phối Glucose theo Outcome")
plt.xlabel("Glucose")
plt.ylabel("Mật độ")
plt.show()
```



Cách đọc Biểu đồ 2 – Histogram/KDE của Glucose theo Outcome

Histogram đọc theo chiều ngang: Trục X là giá trị Glucose; trục Y là mật độ/tần suất theo cấu hình biểu đồ. Mỗi vùng/cột cho biết dữ liệu tập trung nhiều hay ít ở khoảng Glucose đó.

KDE là đường cong: Đường KDE là phiên bản làm trơn của phân phối. Đỉnh cao cho biết vùng giá trị có mật độ quan sát cao. Không nên đọc chiều cao KDE như "số bệnh nhân" tuyệt đối; nó chủ yếu giúp so sánh hình dạng phân phối.

So sánh hai nhóm: Nhìn xem nhóm Outcome=0 và Outcome=1 tập trung ở vùng nào. Nếu đường/phân phối Outcome=1 dịch sang vùng Glucose cao hơn, đó là dấu hiệu Glucose có khả năng phân biệt hai nhóm.

Điểm quan trọng nhất là vùng chồng lấn: Nếu hai phân phối chồng lấn nhiều, có những người có cùng mức Glucose nhưng thuộc hai Outcome khác nhau. Vì vậy Glucose hữu ích nhưng không đủ để đưa ra quyết định hoàn hảo.

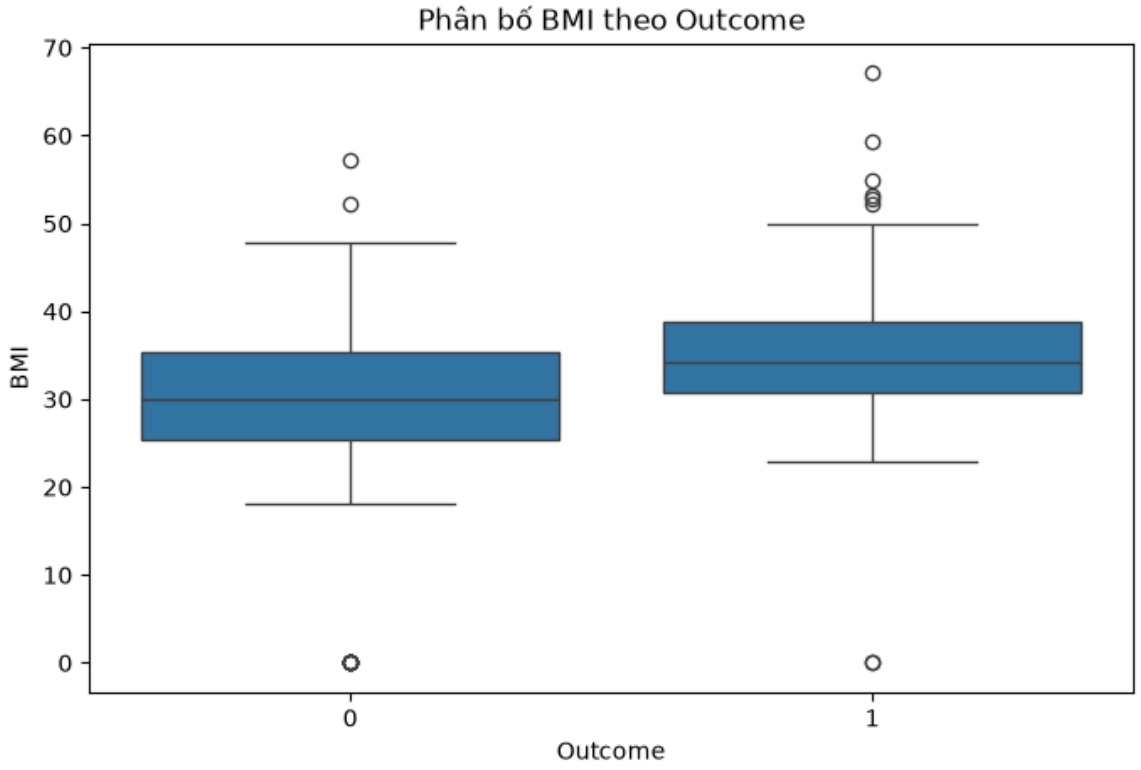
Không được kết luận nhân quả: Biểu đồ cho thấy sự khác biệt phân phối trong dataset, không chứng minh Glucose là nguyên nhân duy nhất gây bệnh.

Liên hệ ML: Feature có khả năng phân biệt tốt có thể giúp model học ranh giới giữa hai lớp; các feature còn lại bổ sung thông tin để xử lý các vùng chồng lấn.

Biểu đồ 3 – BMI theo Outcome

Boxplot cho phép so sánh trung vị, khoảng tứ phân vị và outlier giữa hai lớp. Nếu hai box có vùng chồng lấn lớn, BMI riêng lẻ khó phân loại hoàn hảo và cần kết hợp với các feature khác.

```
# BIỂU ĐỒ 3
plt.figure(figsize=(8,5))
sns.boxplot(data=diabetes, x="Outcome", y="BMI")
plt.title("Phân bố BMI theo Outcome")
plt.xlabel("Outcome")
plt.ylabel("BMI")
plt.show()
```



Cách đọc Biểu đồ 3 – Boxplot BMI theo Outcome

Boxplot cần đọc theo 5 thành phần:

- **Đường giữa hộp:** median (trung vị) – giá trị điển hình ở giữa dữ liệu.
- **Đáy và đỉnh hộp:** Q1 và Q3. Khoảng từ Q1 đến Q3 gọi là **IQR**, chứa 50% dữ liệu trung tâm.
- **Râu (whiskers):** vùng dữ liệu nằm ngoài hộp nhưng chưa được xem là outlier theo quy tắc của thư viện.
- **Các điểm rời:** outlier – những quan sát nằm xa phần lớn dữ liệu.

Cách so sánh: Trước tiên so sánh median của Outcome=0 và Outcome=1. Sau đó so sánh độ cao hộp (IQR) để xem độ phân tán. Cuối cùng kiểm tra outlier.

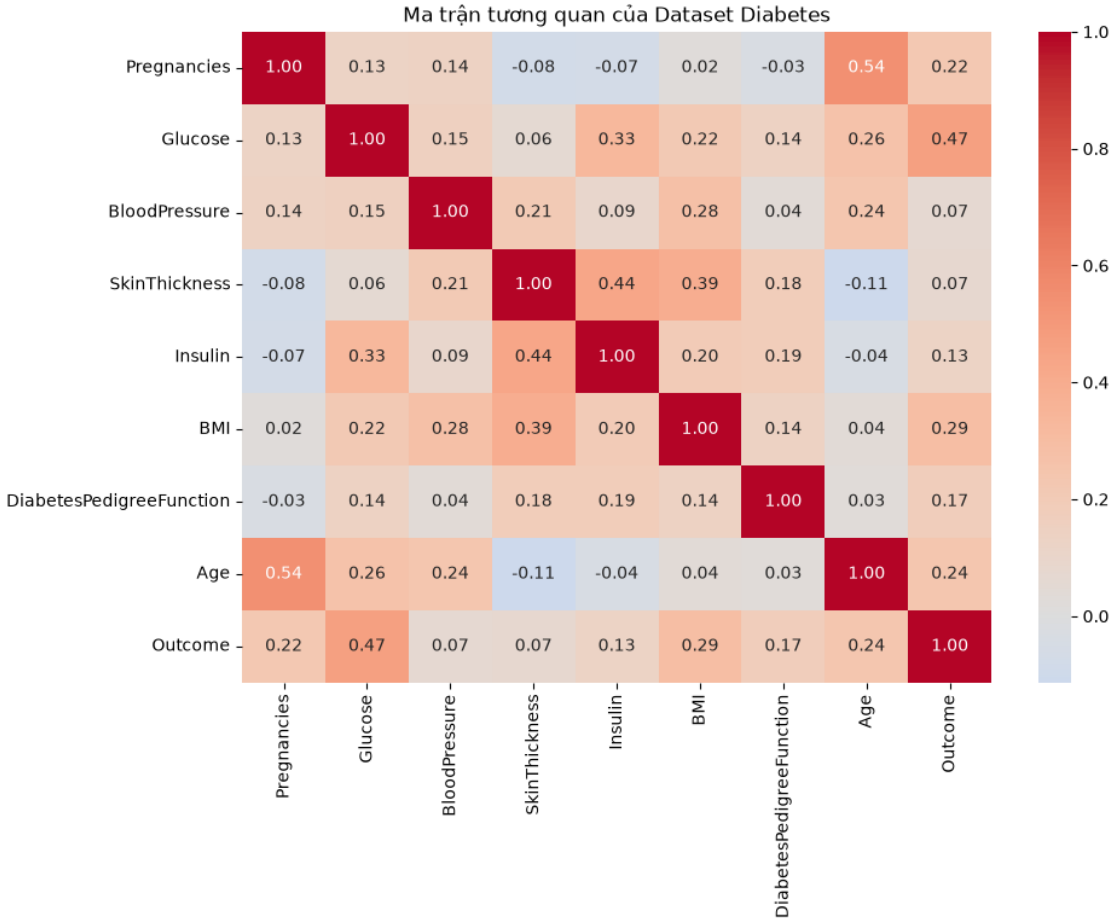
Nếu median của hai nhóm khác nhau nhưng hai hộp vẫn chồng lấn lớn, BMI có thông tin phân biệt nhưng **không thể dùng riêng BMI để phân loại chính xác mọi trường hợp**.

Liên hệ preprocessing: Outlier không tự động có nghĩa là phải xóa. Cần xác định đó là dữ liệu bất thường hay một quan sát thật.

Biểu đồ 4 – Ma trận tương quan

Heatmap giúp quan sát mức độ tương quan tuyến tính giữa các biến số. Tương quan cao không đồng nghĩa với nhân quả. Heatmap chủ yếu hỗ trợ khám phá dữ liệu và phát hiện các quan hệ có thể ảnh hưởng đến mô hình.

```
# BIỂU ĐỒ 4
plt.figure(figsize=(10,7))
sns.heatmap(diabetes.corr(numeric_only=True), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Ma trận tương quan của Dataset Diabetes")
plt.show()
```



Cách đọc Biểu đồ 4 – Correlation Heatmap của Diabetes

Mỗi ô là hệ số tương quan giữa hai biến số, thường nằm trong khoảng **-1 đến +1**:

- Gần **+1**: quan hệ tuyến tính cùng chiều mạnh.
- Gần **0**: quan hệ tuyến tính yếu/không rõ.
- Gần **-1**: quan hệ tuyến tính ngược chiều mạnh.

Cách đọc một ô: Chọn một feature trên hàng và một feature trên cột; giao điểm của chúng cho biết hệ số tương quan. Đường chéo chính thường bằng 1 vì một biến tương quan hoàn hảo với chính nó.

Màu chỉ là cách mã hóa số: Không nên kết luận chỉ vì "màu đậm". Hãy đọc cả giá trị số được ghi trong ô.

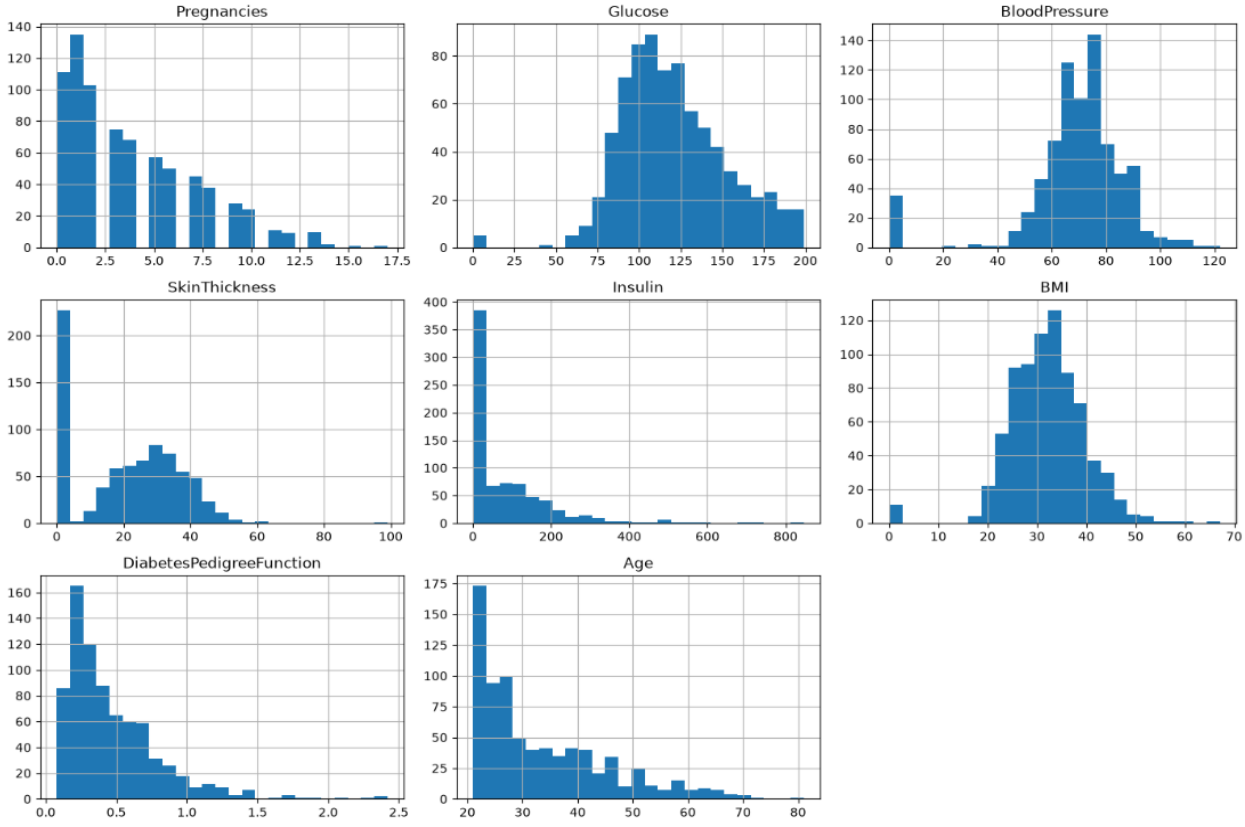
Đặc biệt: Correlation chỉ đo **quan hệ tuyến tính** giữa các biến số. Một quan hệ phi tuyến có thể vẫn tồn tại dù correlation không cao. Correlation cũng không chứng minh nhân quả.

Liên hệ ML: Heatmap giúp phát hiện feature có thông tin liên quan, khả năng dư thừa hoặc phụ thuộc lẫn nhau; nhưng không thể dùng heatmap một mình để quyết định feature importance.

Biểu đồ 5 – Toàn bộ feature

Các histogram giúp quan sát độ lệch, phạm vi và khả năng xuất hiện outlier. Đây là cơ sở để quyết định preprocessing và lý giải tại sao một số mô hình nhạy với scale hơn các mô hình khác.

```
# BIỂU ĐỒ 5
diabetes[feature_cols_d].hist(figsize=(14,10), bins=25)
plt.suptitle("Phân phối các đặc trưng đầu vào", y=1.02)
plt.tight_layout()
plt.show()
```



Cách đọc Biểu đồ 5 – Histogram của toàn bộ feature

Với mỗi histogram, đọc theo 4 bước:

1. **Range:** dữ liệu trải từ giá trị thấp đến cao nào?
2. **Center:** phần lớn quan sát nằm quanh vùng nào?
3. **Shape:** phân phối đối xứng, lệch phải, lệch trái hay có nhiều đỉnh?
4. **Outlier:** có vùng dữ liệu rất thưa hoặc điểm cực trị không?

Ví dụ, nếu histogram bị **lệch phải**, nghĩa là phần lớn dữ liệu nằm ở vùng thấp/trung bình và có một "đuôi" kéo dài về phía giá trị lớn.

Tại sao phải đọc tất cả feature? Vì các model như KNN, SVM và Logistic Regression nhạy với scale hơn Decision Tree/Random Forest. Việc quan sát range và phân phối giúp giải thích tại sao notebook sử dụng preprocessing/standardization.

12. Data Representation và preprocessing

Một số biến trong dataset dùng 0 cho các giá trị không hợp lý về mặt sinh lý, đặc biệt Glucose, BloodPressure, SkinThickness, Insulin, BMI. Trong bài thực hành, các giá trị 0 của những cột này được thay bằng NaN, sau đó **median được học trên training data**.

Để tránh data leakage, việc imputation và scaling nằm trong Pipeline. Do đó fit() của preprocessing chỉ học từ training set.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score, classification_report, roc_curve

diab = diabetes.copy()
zero_as_missing = ["Glucose","BloodPressure","SkinThickness","Insulin","BMI"]
for c in zero_as_missing:
    diab[c] = diab[c].replace(0, np.nan)

X_d = diab[feature_cols_d]
y_d = diab[target_d]

X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(
    X_d, y_d, test_size=0.20, random_state=RANDOM_STATE, stratify=y_d
)
print("Train:", X_train_d.shape, "Test:", X_test_d.shape)
```

```
Train: (614, 8) Test: (154, 8)
```

13. Baseline và 5 mô hình

Baseline sử dụng lớp xuất hiện nhiều nhất. Sau đó huấn luyện 5 mô hình được khuyến nghị trong Assignment.

Điểm quan trọng: tất cả mô hình đều gọi fit(X_train, y_train) và chỉ dùng X_test/y_test để đánh giá.

```python
# HUẤN LUYỆN VÀ ĐÁNH GIÁ 5 MÔ HÌNH
results_d=[]
preds_d={}
probs_d={}

for name, model in models_d.items():
    model.fit(X_train_d, y_train_d)  # fit() bắt buộc: học representation + model từ training data
    pred=model.predict(X_test_d)
    prob=model.predict_proba(X_test_d)[:,1]
    preds_d[name]=pred
    probs_d[name]=prob
    results_d.append({
        "Model":name,
        "Accuracy":accuracy_score(y_test_d,pred),
        "Precision":precision_score(y_test_d,pred,zero_division=0),
        "Recall":recall_score(y_test_d,pred,zero_division=0),
        "F1":f1_score(y_test_d,pred,zero_division=0),
        "ROC-AUC":roc_auc_score(y_test_d,prob)
    })

results_d=pd.DataFrame(results_d).sort_values("F1",ascending=False).reset_index(drop=True)
display(results_d)
```

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	Decision Tree	0.759740	0.639344	0.722222	0.678261	0.762222
1	SVM	0.740260	0.652174	0.555556	0.600000	0.796389
2	KNN	0.727273	0.625000	0.555556	0.588235	0.790000
3	Random Forest	0.733766	0.644444	0.537037	0.585859	0.815833
4	Logistic Regression	0.707792	0.600000	0.500000	0.545455	0.812963

Giải thích kết quả đánh giá

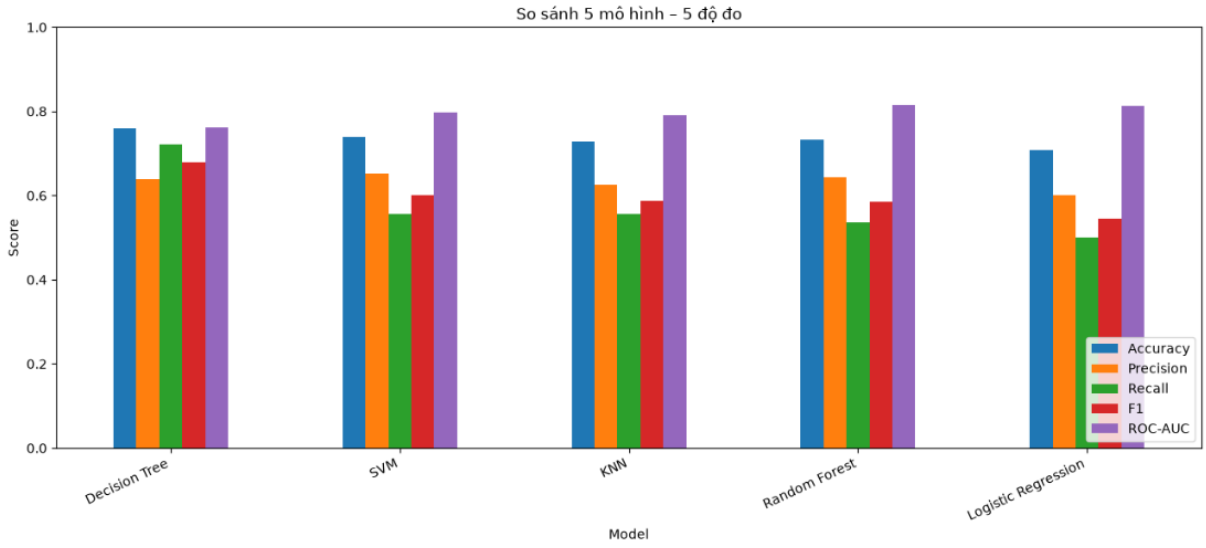
Không chọn mô hình chỉ dựa trên Accuracy. Với bài toán y tế, Recall và F1 có ý nghĩa quan trọng vì hệ thống cần hạn chế bỏ sót trường hợp dương tính nhưng vẫn kiểm soát false positive.

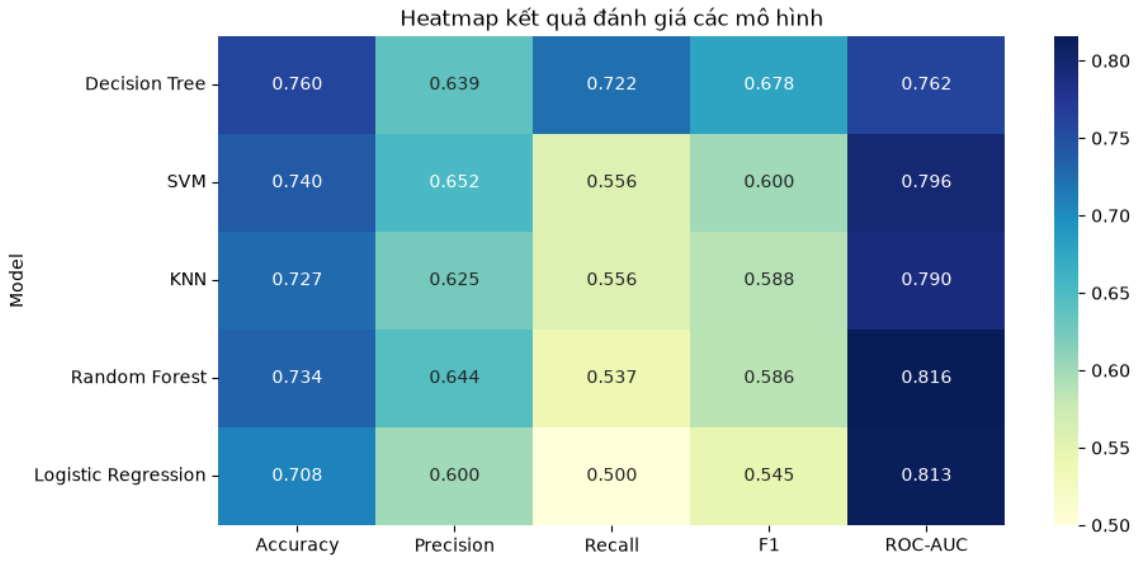
ROC-AUC bổ sung góc nhìn về khả năng xếp hạng hai lớp trên nhiều ngưỡng. Confusion Matrix giúp kiểm tra trực tiếp TP, TN, FP, FN.

Mô hình tốt nhất trong notebook được lựa chọn **theo F1 trên test set trong phần báo cáo minh họa này**; khi sử dụng thực tế cần có validation/cross-validation và đánh giá độc lập.

```python
# BIỂU ĐỒ 6 – SO SÁNH 5 METRICS
plot_d=results_d.set_index("Model")[["Accuracy","Precision","Recall","F1","ROC-AUC"]]
plot_d.plot(kind="bar", figsize=(13,6))
plt.title("So sánh 5 mô hình – 5 độ đo")
plt.ylabel("Score")
plt.ylim(0,1)
plt.xticks(rotation=25, ha="right")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()

# BIỂU ĐỒ 7 – HEATMAP
plt.figure(figsize=(10,5))
sns.heatmap(plot_d, annot=True, fmt=".3f", cmap="YlGnBu")
plt.title("Heatmap kết quả đánh giá các mô hình")
plt.show()
```





Cách đọc Biểu đồ 6 – So sánh 5 model × 5 metric

Đây là biểu đồ **so sánh mô hình**, không phải biểu đồ phân phối. Vì vậy cách đọc khác:

Bước 1: Chọn một metric, ví dụ F1.

Bước 2: So sánh 5 model trên cùng metric đó.

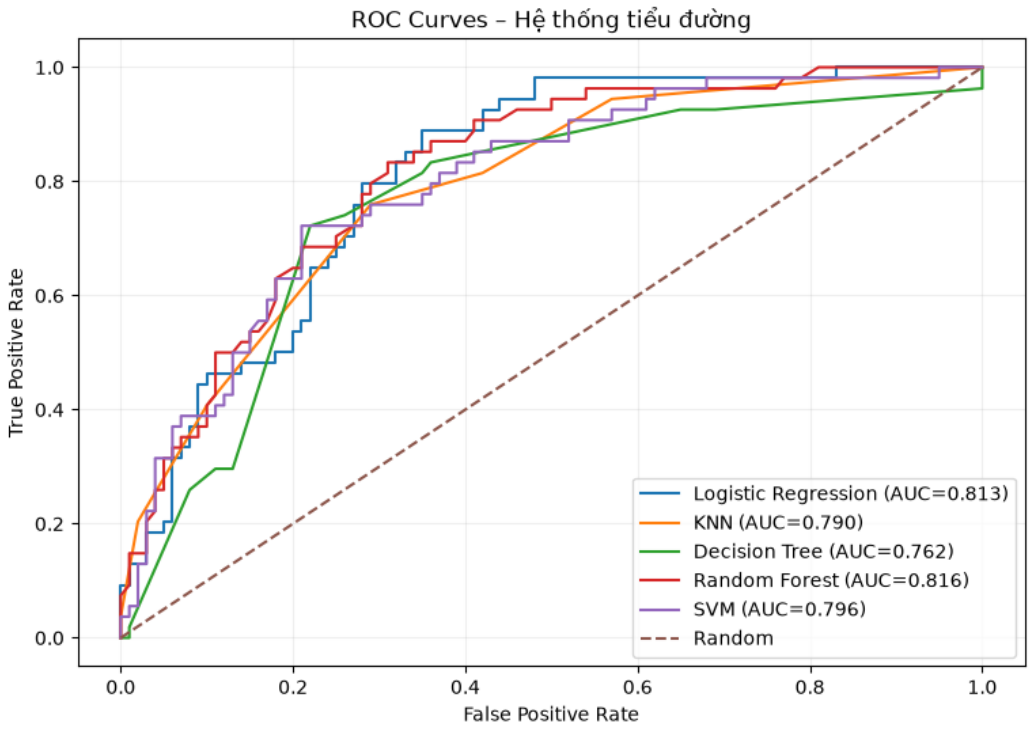
Bước 3: Nhớ hướng tốt/xấu: Accuracy, Precision, Recall, F1, ROC-AUC **càng cao càng tốt**.

Bước 4: Lặp lại với các metric khác.

Nếu một model cao nhất ở Accuracy nhưng thấp hơn rõ rệt ở Recall, không được kết luận model đó tốt nhất ngay. Với bài toán y tế, cần đặt metric phù hợp với mục tiêu hệ thống.

Cách kết luận: "Model A có F1 và Recall cao hơn các model còn lại, trong khi Accuracy cũng cạnh tranh; do mục tiêu giảm bỏ sót positive, A là ứng viên phù hợp hơn." Kết luận phải dựa trên bảng số liệu chứ không chỉ nhìn màu/cột.

```
# BIỂU ĐỒ 8 - ROC CURVES
plt.figure(figsize=(9,6))
for name, prob in probs_d.items():
    fpr,tpr,_=roc_curve(y_test_d,prob)
    auc=roc_auc_score(y_test_d,prob)
    plt.plot(fpr,tpr,label=f"{name} (AUC={auc:.3f})")
plt.plot([0,1],[0,1],"--",label="Random")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves - Hệ thống tiểu đường")
plt.legend()
plt.grid(alpha=.2)
plt.show()
```



Cách đọc Biểu đồ ROC Curve

ROC có:

- Trục X = **False Positive Rate (FPR)** = FP/(FP+TN).
- Trục Y = **True Positive Rate (TPR/Recall)** = TP/(TP+FN).

Mỗi điểm trên đường ROC tương ứng với một **ngưỡng dự đoán** khác nhau. Khi thay đổi threshold, TPR và FPR thay đổi.

Đường chéo từ góc dưới trái lên góc trên phải biểu diễn mức gần với dự đoán ngẫu nhiên. Đường ROC càng nằm phía trên và bên trái, model càng có khả năng phân biệt hai lớp.

AUC: diện tích dưới đường ROC; càng gần 1 càng tốt. AUC không phải "độ chính xác y khoa", mà là khả năng xếp hạng/phân biệt hai lớp trên nhiều threshold.

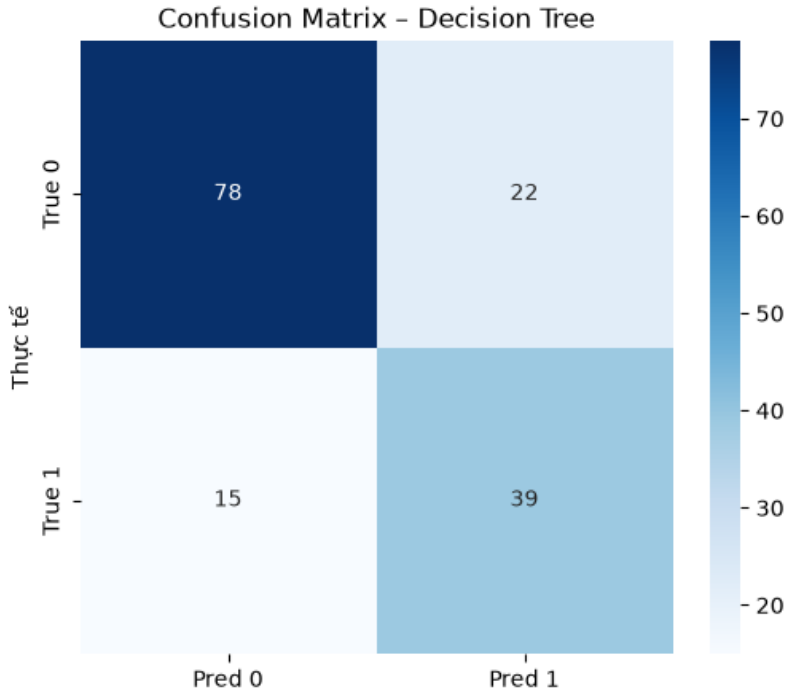
Cách đọc khi có nhiều model: tìm đường cong nằm cao hơn phần lớn diện tích. Tuy nhiên nếu hai đường rất gần nhau, cần quay lại bảng metric để xem chênh lệch thực tế.

Giải thích ROC Curve

Đường ROC biểu diễn sự đánh đổi giữa True Positive Rate và False Positive Rate khi thay đổi ngưỡng dự đoán. AUC càng lớn thì khả năng phân biệt hai lớp càng tốt trên toàn bộ các ngưỡng.

Không nên diễn giải AUC là "độ chính xác y khoa"; nó chỉ là một chỉ số đánh giá trên dataset.

```
# BIỂU ĐỒ 9 – CONFUSION MATRIX CỦA MÔ HÌNH TỐT NHẤT
best_d_name=results_d.iloc[0]["Model"]
cm=confusion_matrix(y_test_d,preds_d[best_d_name])
plt.figure(figsize=(6,5))
sns.heatmap(cm,annot=True,fmt="d",cmap="Blues",xticklabels=["Pred 0","Pred 1"],yticklabels=["True 0","True 1"])
plt.title(f"Confusion Matrix – {best_d_name}")
plt.xlabel("Dự đoán")
plt.ylabel("Thực tế")
plt.show()
print(classification_report(y_test_d,preds_d[best_d_name],zero_division=0))
```



	precision	recall	f1-score	support
0	0.84	0.78	0.81	100
1	0.64	0.72	0.68	54
accuracy			0.76	154
macro avg	0.74	0.75	0.74	154
weighted avg	0.77	0.76	0.76	154

Cách đọc Biểu đồ 9 – Confusion Matrix

Confusion Matrix là biểu đồ cần đọc **từng ô**, không đọc theo màu trước. Với bài toán nhị phân:

- **TN:** thực tế 0, dự đoán 0 → đúng.
- **FP:** thực tế 0, dự đoán 1 → báo động nhầm.
- **FN:** thực tế 1, dự đoán 0 → **bỏ sót trường hợp dương tính**.
- **TP:** thực tế 1, dự đoán 1 → phát hiện đúng.

Thông thường hàng là **Actual/True**, cột là **Predicted**; hãy nhìn nhãn trục của chính hình để xác nhận.

Đọc theo bài toán y tế: FN thường đặc biệt đáng chú ý vì hệ thống đã dự đoán "không nguy cơ" cho một trường hợp thực tế là dương tính. Tuy nhiên FP cũng có chi phí vì có thể khiến người dùng phải kiểm tra thêm.

Từ 4 ô này suy ra Precision, Recall, F1 và Accuracy. Vì vậy Confusion Matrix giúp giải thích **model sai như thế nào**, trong khi một con số Accuracy chỉ nói tổng thể đúng bao nhiêu.

14. Experiment 1 – Model Comparison

Câu hỏi: Trong cùng điều kiện train/test và preprocessing, mô hình nào đạt kết quả tốt hơn?

Kết quả được lưu ở results_d. Việc lựa chọn mô hình phải dựa trên bảng metric và confusion matrix, không dựa vào cảm tính.

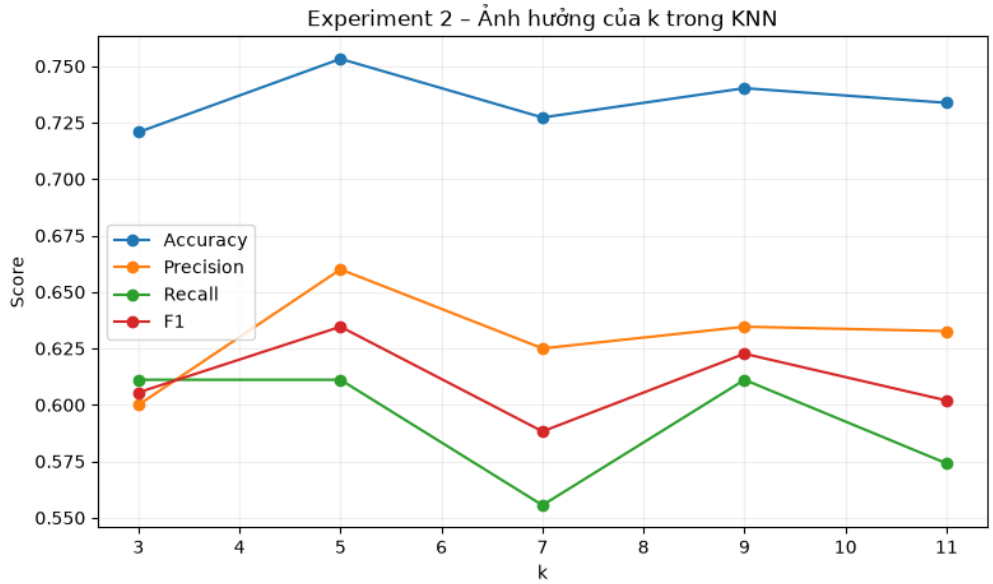
15. Experiment 2 – Hyperparameter Investigation

Câu hỏi: Giá trị k của KNN ảnh hưởng thế nào đến F1 và Recall?

Chỉ thay đổi một hyperparameter là n_neighbors; các điều kiện khác giữ nguyên.

```
knn_exp=[]
for k in [3,5,7,9,11]:
    model=classification_pipeline(KNeighborsClassifier(n_neighbors=k))
    model.fit(X_train_d,y_train_d)
    pred=model.predict(X_test_d)
    knn_exp.append({"k":k,
                    "Accuracy":accuracy_score(y_test_d,pred),
                    "Precision":precision_score(y_test_d,pred,zero_division=0),
                    "Recall":recall_score(y_test_d,pred,zero_division=0),
                    "F1":f1_score(y_test_d,pred,zero_division=0)})
knn_exp=pd.DataFrame(knn_exp)
display(knn_exp)
knn_exp.set_index("k")[["Accuracy","Precision","Recall","F1"]].plot(marker="o",figsize=(9,5))
plt.title("Experiment 2 - Ảnh hưởng của k trong KNN")
plt.xlabel("k")
plt.ylabel("Score")
plt.grid(alpha=.2)
plt.show()
```

	k	Accuracy	Precision	Recall	F1
0	3	0.720779	0.600000	0.611111	0.605505
1	5	0.753247	0.660000	0.611111	0.634615
2	7	0.727273	0.625000	0.555556	0.588235
3	9	0.740260	0.634615	0.611111	0.622642
4	11	0.733766	0.632653	0.574074	0.601942



Nhận xét: thay đổi k làm thay đổi cách mô hình cân bằng giữa tính cục bộ và tính ổn định. k nhỏ nhạy với nhiễu; k lớn làm mô hình mượt hơn nhưng có thể mất các cấu trúc cục bộ. Ta chọn giá trị phù hợp dựa trên metric, không chọn tùy ý.

16. Experiment 3 – Representation Investigation

Câu hỏi: Chuẩn hóa feature có giúp các mô hình nhạy với scale như KNN và SVM hay không?

Ta so sánh KNN có StandardScaler và KNN không scaling, trong khi cùng imputation và cùng train/test split.

```
rep_rows=[]
for scaled in [True,False]:
    m=classification_pipeline(KNeighborsClassifier(n_neighbors=7),scale=scaled)
    m.fit(X_train_d,y_train_d)
    pred=m.predict(X_test_d)
    rep_rows.append({"Representation":"Standardized" if scaled else "Unscaled",
                     "Accuracy":accuracy_score(y_test_d,pred),
                     "Precision":precision_score(y_test_d,pred,zero_division=0),
                     "Recall":recall_score(y_test_d,pred,zero_division=0),
                     "F1":f1_score(y_test_d,pred,zero_division=0)})
rep_df=pd.DataFrame(rep_rows)
display(rep_df)
```

	Representation	Accuracy	Precision	Recall	F1
0	Standardized	0.727273	0.625000	0.555556	0.588235
1	Unscaled	0.675325	0.541667	0.481481	0.509804

Kết luận Experiment 3

KNN sử dụng khoảng cách nên scale của feature ảnh hưởng trực tiếp đến khoảng cách. Standardization đưa các feature về scale tương đối đồng đều và thường giúp KNN hoạt động ổn định hơn. Đây là minh họa trực tiếp cho nguyên lý của Assignment: **thay đổi representation có thể làm thay đổi kết quả học**.

17. Final Model và lưu model

Sau khi hoàn thành thí nghiệm, chọn mô hình tốt nhất theo tiêu chí đã công bố. Final model được fit lại trên **toàn bộ training set**, không sử dụng test set để fit.

Model được lưu kèm preprocessing bằng joblib, nhờ đó application nhận dữ liệu thô và áp dụng đúng representation như lúc training.

```
best_d_name=results_d.iloc[0]["Model"]
final_diabetes_model=models_d[best_d_name]

# fit() lại final model trên training set
final_diabetes_model.fit(X_train_d,y_train_d)

os.makedirs("artifacts",exist_ok=True)
joblib.dump(final_diabetes_model,"artifacts/diabetes_final_model.joblib")

with open("artifacts/diabetes_model_info.json","w",encoding="utf-8") as f:
    json.dump({
        "model":best_d_name,
        "features":feature_cols_d,
        "target":"Outcome",
        "note":"Educational screening system; not a medical diagnosis."
    },f,ensure_ascii=False,indent=2)

print("Final model:",best_d_name)
print("Đã lưu artifacts/diabetes_final_model.joblib")
```

PHẦN III – HỆ THỐNG 2: DỰ ĐOÁN GIÁ NHÀ

18. Mục tiêu

Hệ thống nhận các thuộc tính của bất động sản và dự đoán Price.

Đây là bài toán **Regression**, vì target là một đại lượng số. Theo Assignment, các metric phù hợp gồm MAE, MSE, RMSE và R^2.

```
# LOAD DATASET GIÁ NHÀ
HOUSING_PATH=Path(r"C:\DATA\vietnam_housing_dataset.csv")
HOUSING_CSV=resolve_csv(HOUSING_PATH, "/mnt/data/vietnam_housing_dataset(3).csv")
housing=pd.read_csv(HOUSING_CSV)

print("Đường dẫn:",HOUSING_CSV)
print("Kích thước:",housing.shape)
display(housing.head())
display(pd.DataFrame({
    "dtype":housing.dtypes.astype(str),
    "missing":housing.isna().sum(),
    "unique":housing.nunique()
}))
```

Mô tả dataset

Dataset có các thuộc tính bất động sản gồm: Address, Area, Frontage, Access Road, House direction, Balcony direction, Floors, Bedrooms, Bathrooms, Legal status, Furniture state và target Price.

Có cả biến numerical và categorical, đồng thời một số cột có missing. Vì vậy representation của mô hình phải xử lý cả hai loại dữ liệu.

```
feature_cols_h=[c for c in housing.columns if c!="Price"]
target_h="Price"
num_h=housing[feature_cols_h].select_dtypes(include=np.number).columns.tolist()
cat_h=housing[feature_cols_h].select_dtypes(exclude=np.number).columns.tolist()

print("Numerical features:",num_h)
print("Categorical features:",cat_h)
print("Target:",target_h)
print(housing[target_h].describe())
```

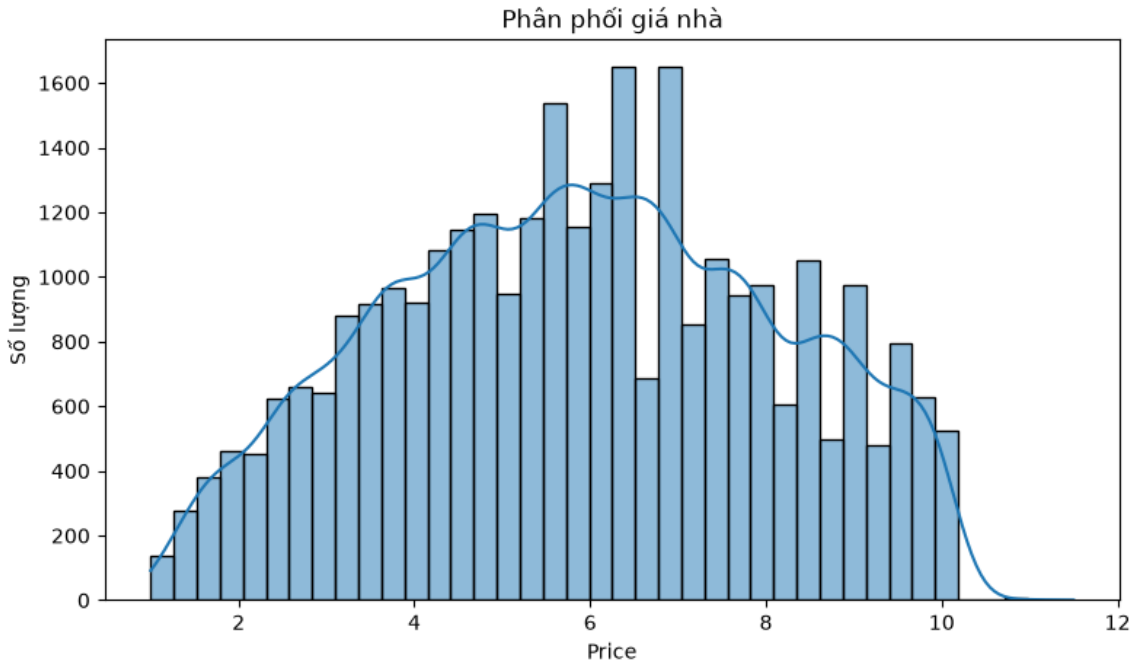
```
Numerical features: ['Area', 'Frontage', 'Access Road', 'Floors', 'Bedrooms', 'Bathrooms']
Categorical features: ['Address', 'House direction', 'Balcony direction', 'Legal status', 'Furniture state']
Target: Price
count    30229.000000
mean         5.872078
std          2.211877
min          1.000000
25%          4.200000
50%          5.900000
75%          7.500000
max         11.500000
Name: Price, dtype: float64
```

19. EDA – Biểu đồ giá nhà và giải thích

Biểu đồ 1 – Phân phối Price

Giá nhà thường có thể lệch phải do một số bất động sản có giá rất cao. Vì vậy MAE/RMSE nên được xem cùng nhau; RMSE nhạy hơn với các sai số lớn.

```
plt.figure(figsize=(9,5))
sns.histplot(housing["Price"],kde=True,bins=40)
plt.title("Phân phối giá nhà")
plt.xlabel("Price")
plt.ylabel("Số lượng")
plt.show()
```



Cách đọc Biểu đồ 1 – Histogram phân phối Price

Trục X là **giá nhà**, trục Y là số lượng/mật độ quan sát tùy cấu hình. Mỗi cột đại diện cho một khoảng giá.

Nếu histogram lệch phải: phần lớn nhà nằm ở mức giá thấp/trung bình, nhưng có một số nhà rất đắt tạo thành đuôi dài bên phải.

Nếu có cột rất cao: nhiều quan sát tập trung trong khoảng giá đó.

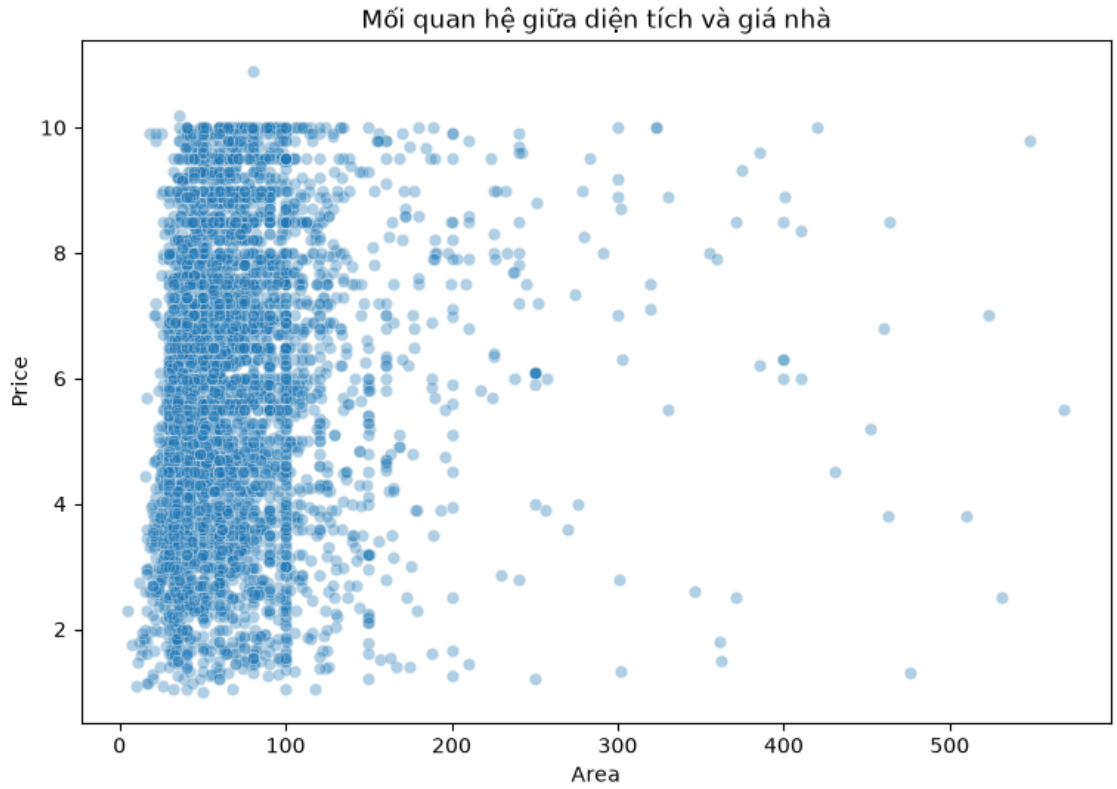
Nếu có đuôi dài/outlier: các sai số lớn ở nhóm nhà giá cao có thể ảnh hưởng mạnh đến MSE/RMSE. Đây là lý do nên đọc MAE và RMSE cùng nhau.

Histogram không cho biết model dự đoán tốt hay xấu; nó mô tả **target trước khi học** và giúp hiểu tính chất của bài toán regression.

Biểu đồ 2 – Area và Price

Scatter plot giúp quan sát xu hướng giữa diện tích và giá. Nếu xu hướng tăng nhưng có độ phân tán lớn, diện tích là thông tin hữu ích nhưng không đủ để dự đoán giá; vị trí, pháp lý, hướng và tiện nghi cũng có thể đóng vai trò.

```
plt.figure(figsize=(9,6))
sns.scatterplot(data=housing.sample(min(5000,len(housing)),random_state=RANDOM_STATE),
                x="Area",y="Price",alpha=.35)
plt.title("Mối quan hệ giữa diện tích và giá nhà")
plt.xlabel("Area")
plt.ylabel("Price")
plt.show()
```



Cách đọc Biểu đồ 2 – Scatter Area và Price

Mỗi **chấm** đại diện cho một căn nhà.

- Trục X = Area.

- Trục Y = Price.

Đọc hướng: nếu đám mây điểm có xu hướng đi từ trái dưới lên phải trên, diện tích và giá có xu hướng tăng cùng nhau.

Đọc độ phân tán: nếu tại cùng một Area có nhiều mức Price rất khác nhau, Area không đủ để giải thích giá. Khi đó các yếu tố như vị trí, pháp lý, số tầng, phòng ngủ, hướng... có thể bổ sung thông tin.

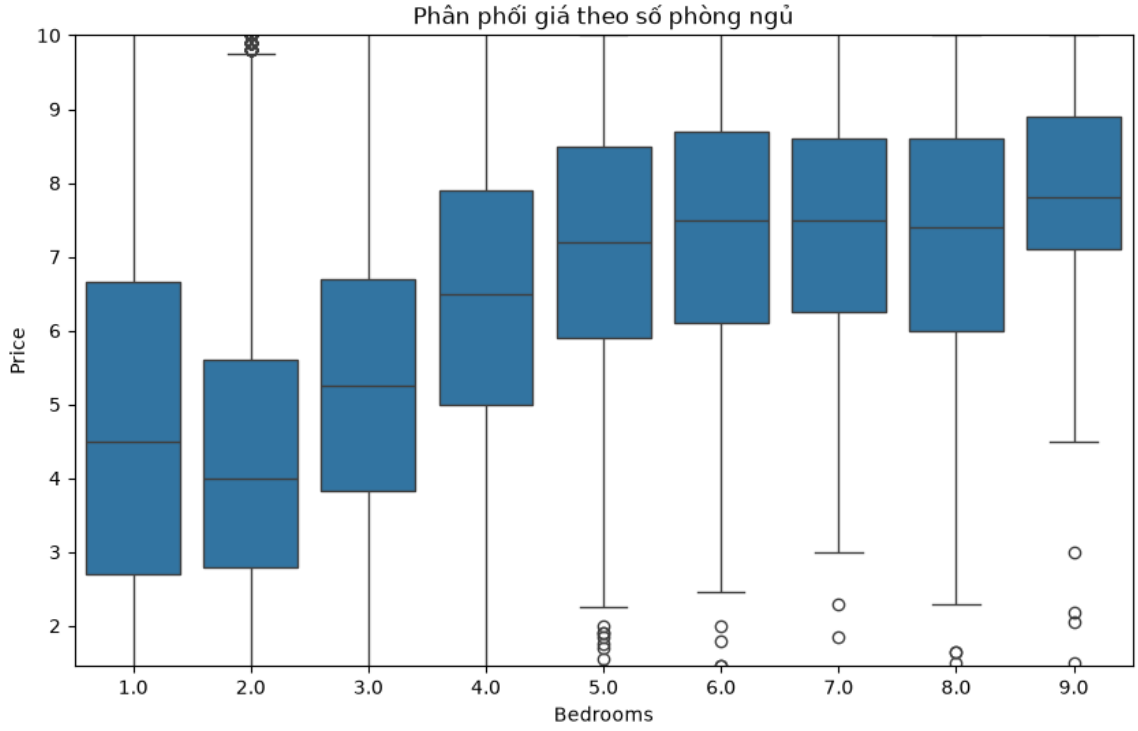
Đọc outlier: điểm nằm rất xa phần lớn đám mây có thể là nhà có đặc điểm đặc biệt hoặc dữ liệu bất thường. Không tự động xóa chúng.

Lưu ý: scatter chỉ cho thấy association/trend trong dataset, không chứng minh "diện tích gây ra giá".

Biểu đồ 3 – Price theo Bedrooms

Boxplot cho phép so sánh phân phối giá giữa các nhóm số phòng ngủ. Chênh lệch giữa các nhóm gợi ý rằng Bedrooms mang thông tin dự đoán, nhưng cũng cần lưu ý số phòng ngủ có thể liên quan đến diện tích và vị trí.

```
plt.figure(figsize=(10,6))
sns.boxplot(data=housing,x="Bedrooms",y="Price")
plt.title("Phân phối giá theo số phòng ngủ")
plt.xlabel("Bedrooms")
plt.ylabel("Price")
plt.ylim(housing["Price"].quantile(.01),housing["Price"].quantile(.99))
plt.show()
```



Cách đọc Biểu đồ 3 – Boxplot Price theo Bedrooms

Mỗi box tương ứng với một nhóm số phòng ngủ. Đọc từng nhóm theo:

1. **Median:** đường giữa hộp → giá điển hình của nhóm.
2. **IQR:** chiều cao hộp → mức độ phân tán của 50% dữ liệu trung tâm.
3. **Whiskers:** phạm vi dữ liệu không bị xem là outlier.
4. **Outlier:** điểm nằm xa hộp/râu.

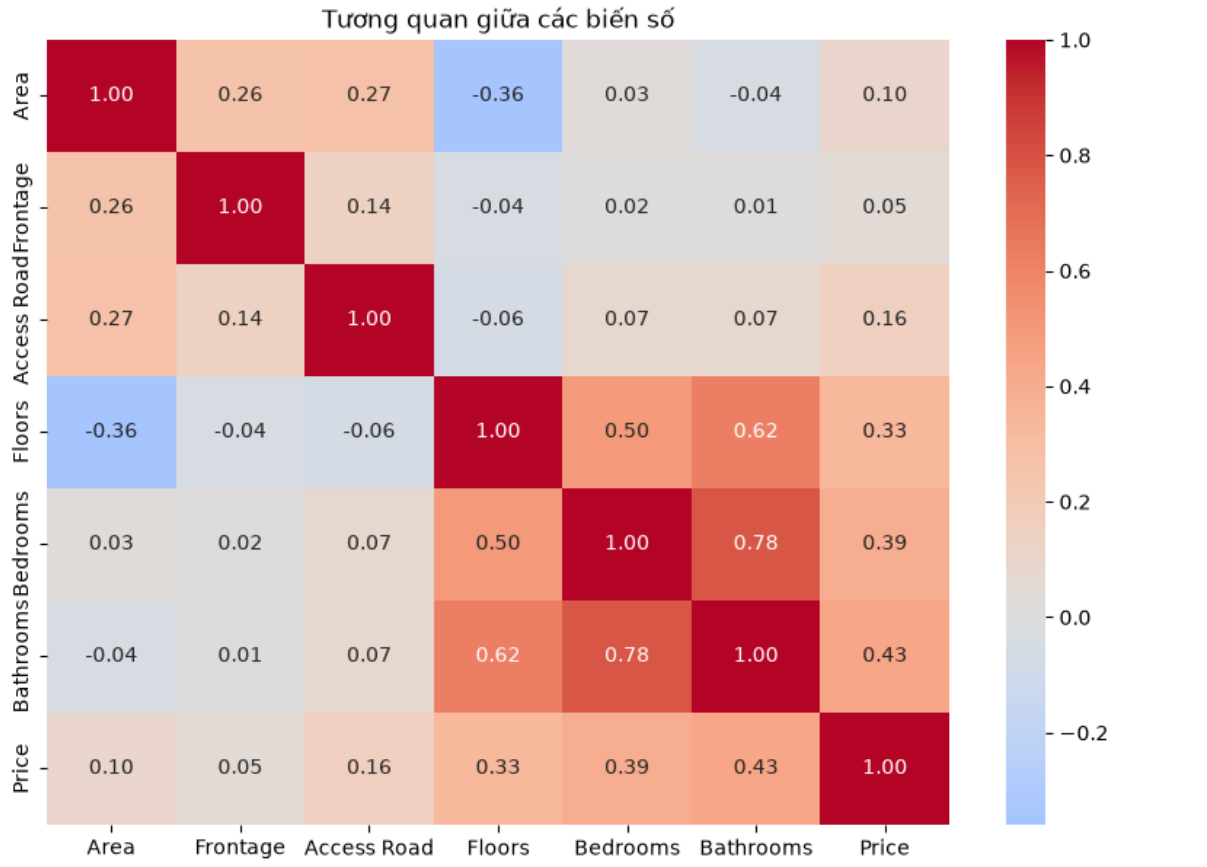
So sánh các nhóm: nhóm nào có median cao hơn thì có giá điển hình cao hơn trong dataset. Nếu hộp chồng lấn nhiều, số phòng ngủ một mình không đủ để phân biệt giá.

Một cảnh báo quan trọng: số phòng ngủ có thể liên quan với diện tích. Vì vậy không nên diễn giải chênh lệch giá theo Bedrooms như một quan hệ nhân quả đơn giản.

Biểu đồ 4 – Correlation của biến số

Heatmap chỉ xét biến số và hỗ trợ phát hiện quan hệ tuyến tính. Vì target Price có thể chịu tác động phi tuyến và từ categorical features, correlation không đủ để kết luận feature importance.

```
plt.figure(figsize=(10,7))
sns.heatmap(housing.select_dtypes(include=np.number).corr(),annot=True,fmt=".2f",cmap="coolwarm",center=0)
plt.title("Tương quan giữa các biến số")
plt.show()
```



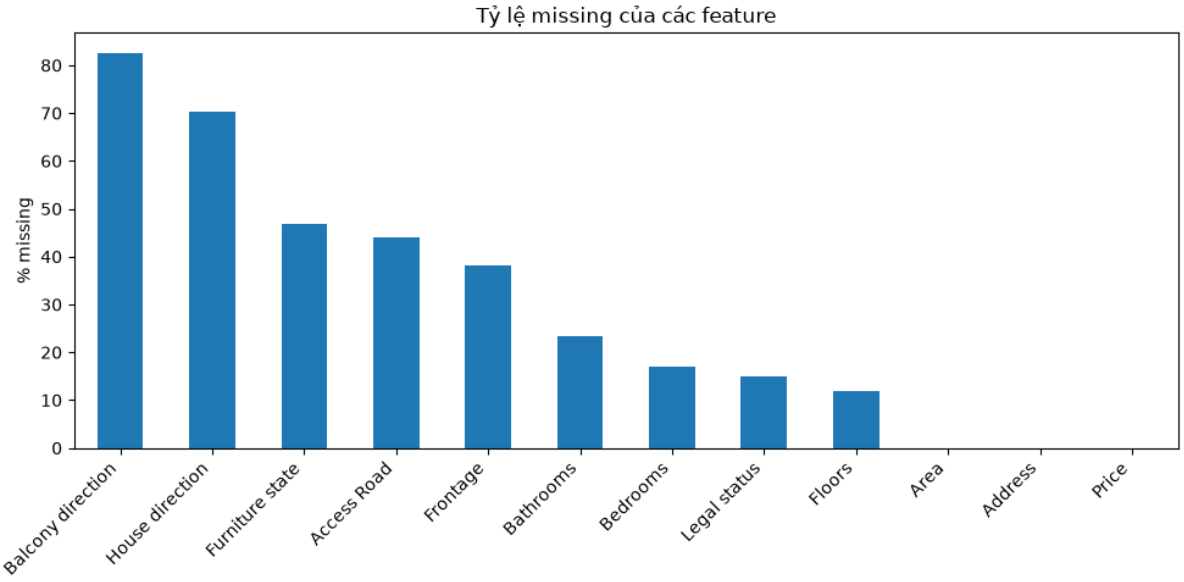
Cách đọc Biểu đồ 4 – Correlation Heatmap của dữ liệu nhà

Cách đọc giống heatmap của Diabetes: xem **giá trị hệ số**, dấu (+/-) và độ lớn.

Ở đây cần đặc biệt chú ý ô giao giữa feature và Price. Hệ số dương lớn cho thấy xu hướng tuyến tính cùng chiều; hệ số âm cho thấy xu hướng ngược chiều.

Tuy nhiên, dữ liệu nhà có nhiều biến categorical. Heatmap correlation chỉ xử lý biến số nên **không thể đại diện cho toàn bộ thông tin của dataset**. Đây là lý do representation cuối cùng sử dụng cả numerical và categorical features.

```
# BIỂU ĐỒ 5 – MISSING VALUE
missing_h=housing.isna().mean().sort_values(ascending=False)*100
plt.figure(figsize=(10,5))
missing_h.plot(kind="bar")
plt.title("Tỷ lệ missing của các feature")
plt.ylabel("% missing")
plt.xticks(rotation=45,ha="right")
plt.tight_layout()
plt.show()
```



Cách đọc Biểu đồ 5 – Tỷ lệ Missing Values

Trục X là tên feature; trục Y là **tỷ lệ phần trăm giá trị thiếu**.

Cột càng cao → feature càng thiếu dữ liệu.

Ví dụ, nếu một feature có 20% missing, có nghĩa khoảng 20% quan sát không có giá trị ở feature đó; không có nghĩa feature "sai 20%".

Liên hệ preprocessing: notebook không xóa toàn bộ dòng chỉ vì có missing, vì có thể mất rất nhiều dữ liệu. Thay vào đó, numerical được impute bằng median và categorical bằng most frequent trong Pipeline.

Điểm quan trọng: preprocessing phải được **fit trên training data**, sau đó áp dụng cho test/new input để tránh data leakage.

20. Train/Test và Representation cho giá nhà

```
# FIT() VÀ ĐÁNH GIÁ 5 MÔ HÌNH HỒI QUY
results_h=[]
preds_h={}
for name,model in models_h.items():
    model.fit(X_train_h,y_train_h)  # fit() toàn bộ preprocessing + model trên train
    pred=model.predict(X_test_h)
    preds_h[name]=pred
    results_h.append({
        "Model":name,
        "MAE":mean_absolute_error(y_test_h,pred),
        "MSE":mean_squared_error(y_test_h,pred),
        "RMSE":mean_squared_error(y_test_h,pred)**.5,
        "R2":r2_score(y_test_h,pred),
        "MedianAE":median_absolute_error(y_test_h,pred)
    })
results_h=pd.DataFrame(results_h).sort_values(["R2","MAE"],ascending=[False,True]).reset_index(drop=True)
display(results_h)
```

	Model	MAE	MSE	RMSE	R2	MedianAE
0	Random Forest	1.404032	3.148217	1.774322	0.354333	1.181063
1	Ridge Regression	1.424061	3.168052	1.779902	0.350265	1.215548
2	Gradient Boosting	1.447967	3.220464	1.794565	0.339516	1.254850
3	Linear Regression	1.266838	3.518467	1.875758	0.278398	0.862785
4	Decision Tree	1.490221	3.554503	1.885339	0.271008	1.251600

Giải thích kết quả

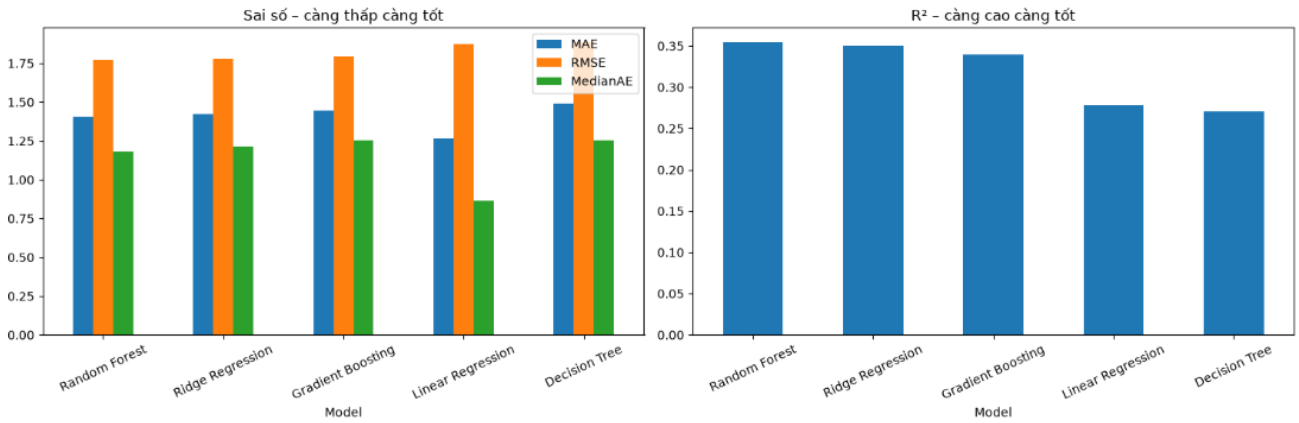
Đối với regression:

- MAE, MSE, RMSE và Median Absolute Error: **càng thấp càng tốt**.
- R^2: **càng cao càng tốt**.

RMSE phạt mạnh các lỗi lớn nên hữu ích khi các trường hợp dự đoán sai rất xa giá trị thực cần được chú ý. MAE dễ diễn giải hơn. MedianAE ít bị ảnh hưởng bởi một số outlier cực lớn.

Mô hình tốt nhất được lựa chọn bằng kết hợp R^2 cao và sai số thấp, đồng thời phải xét mục tiêu ứng dụng.

```
# BIỂU ĐỒ 6 – SO SÁNH 5 MÔ HÌNH
display_df=results_h.set_index("Model")[["MAE","RMSE","R2","MedianAE"]]
fig,axs=plt.subplots(1,2,figsize=(15,5))
display_df[["MAE","RMSE","MedianAE"]].plot(kind="bar",ax=axs[0])
axs[0].set_title("Sai số – càng thấp càng tốt")
axs[0].tick_params(axis="x",rotation=25)
display_df[["R2"]].plot(kind="bar",ax=axs[1],legend=False)
axs[1].set_title("R² – càng cao càng tốt")
axs[1].tick_params(axis="x",rotation=25)
plt.tight_layout()
plt.show()
```



Cách đọc Biểu đồ 6 – So sánh mô hình Regression

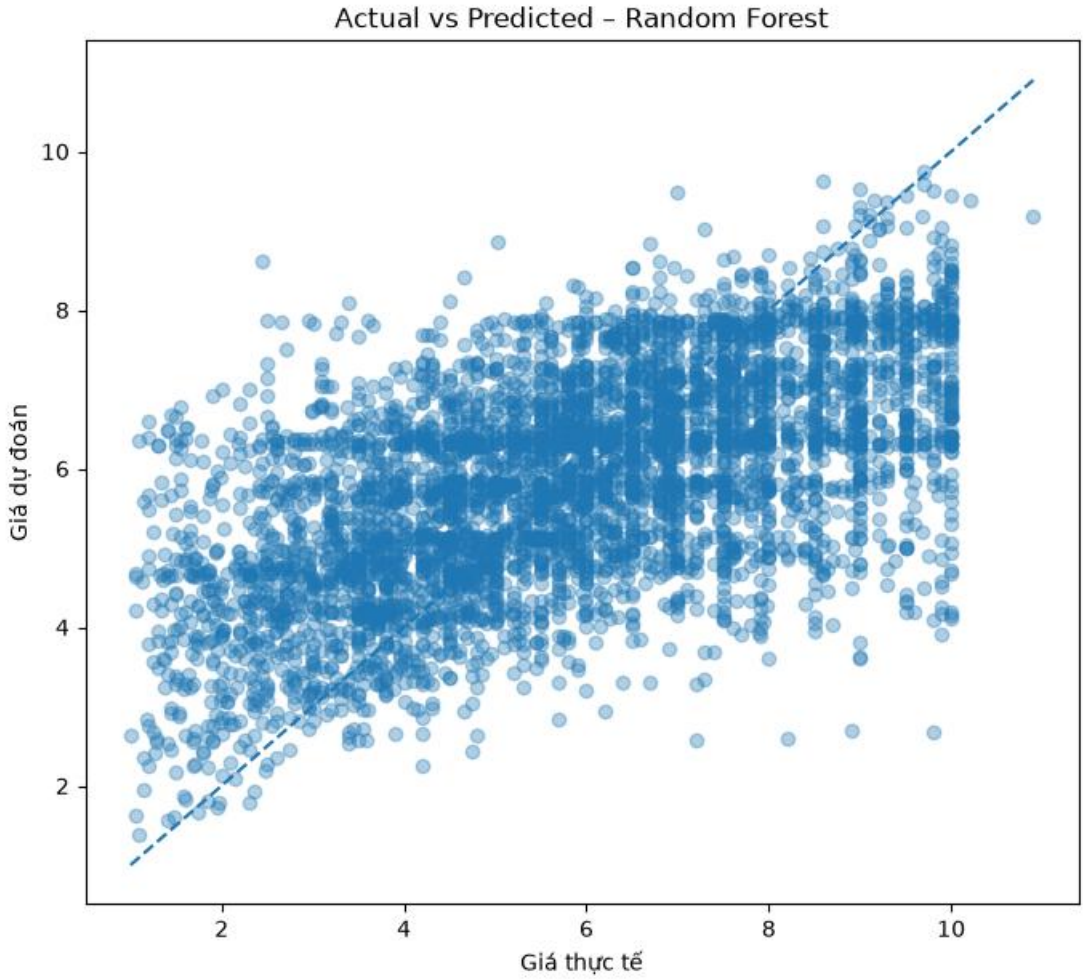
Biểu đồ này cần đọc theo từng metric:

- **MAE:** thấp hơn tốt hơn.
- **RMSE:** thấp hơn tốt hơn; nhạy với sai số rất lớn.
- **MedianAE:** thấp hơn tốt hơn.
- **R^2:** cao hơn tốt hơn.

Không nên nhìn "cột cao nhất" chung cho tất cả metric vì hướng tốt/xấu khác nhau. Ví dụ R^2 cao là tốt nhưng RMSE cao là xấu.

Cách kết luận: ưu tiên metric phù hợp mục tiêu, sau đó kiểm tra các metric còn lại để tránh chọn một model chỉ tốt ở một khía cạnh.

```
# BIỂU ĐỒ 7 – ACTUAL VS PREDICTED CỦA MODEL TỐT NHẤT
best_h_name=results_h.iloc[0]["Model"]
pred_best_h=preds_h[best_h_name]
sample_idx=np.random.RandomState(RANDOM_STATE).choice(len(y_test_h),size=min(5000,len(y_test_h)),replace=False)
plt.figure(figsize=(8,7))
plt.scatter(y_test_h.iloc[sample_idx],pred_best_h[sample_idx],alpha=.35)
lims=[min(y_test_h.min(),pred_best_h.min()),max(y_test_h.max(),pred_best_h.max())]
plt.plot(lims,lims,"--")
plt.xlabel("Giá thực tế")
plt.ylabel("Giá dự đoán")
plt.title(f"Actual vs Predicted – {best_h_name}")
plt.show()
```



Cách đọc Biểu đồ 7 – Actual vs Predicted

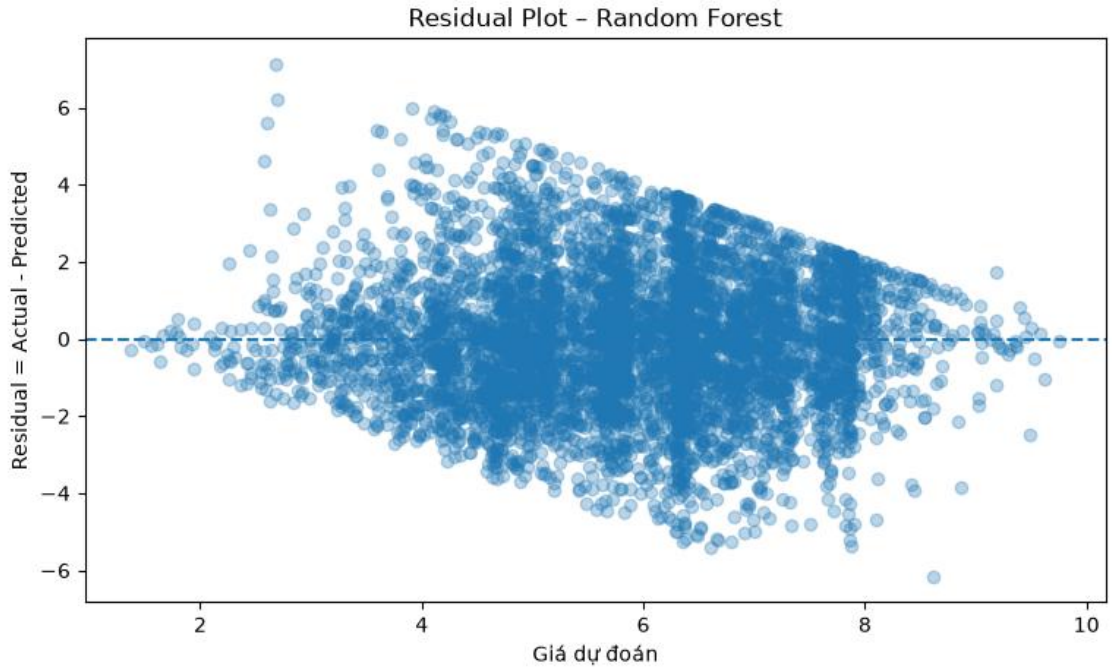
- Trục X = **Actual Price**: giá thật trong test set.
- Trục Y = **Predicted Price**: giá model dự đoán.
- Đường y=x = dự đoán hoàn hảo.

Nếu một điểm nằm trên đường, model dự đoán cao hơn giá thật; nếu nằm dưới, model dự đoán thấp hơn giá thật. Điểm càng xa đường y=x, sai số càng lớn.

Điều cần tìm: một đám mây điểm bám tương đối sát đường chéo là dấu hiệu tốt. Nếu các điểm cong, tạo dải hoặc phân tán tăng mạnh ở vùng giá cao, model còn có sai số có cấu trúc hoặc phương sai sai số thay đổi.

Biểu đồ này rất hữu ích để hiểu **model sai ở đâu**, nhưng vẫn phải dùng MAE/RMSE/R^2 để định lượng.

```python
# BIỂU ĐỒ 8 – RESIDUAL
residual=y_test_h.to_numpy()-pred_best_h
plt.figure(figsize=(9,5))
plt.scatter(pred_best_h,residual,alpha=.3)
plt.axhline(0,linestyle="--")
plt.xlabel("Giá dự đoán")
plt.ylabel("Residual = Actual - Predicted")
plt.title(f"Residual Plot – {best_h_name}")
plt.show()
```



Cách đọc Biểu đồ 8 – Residual Plot

Residual được tính:

Residual = Actual - Predicted

- Residual = 0 → dự đoán đúng.
- Residual > 0 → model dự đoán thấp hơn thực tế (under-predict).
- Residual < 0 → model dự đoán cao hơn thực tế (over-predict).

Trục X: giá dự đoán. **Trục Y:** residual.

Một residual plot tốt thường có các điểm phân tán quanh đường 0, không tạo pattern rõ ràng.

Nếu residual tạo hình cong → quan hệ phi tuyến có thể chưa được mô hình hóa tốt. Nếu độ rộng residual tăng theo X → có thể có heteroscedasticity, tức sai số lớn dần theo mức giá. Nếu nhiều điểm nằm hẳn trên hoặc dưới 0 ở một vùng → model có bias ở vùng đó.

Residual plot vì vậy giúp trả lời câu hỏi: **“Model không chỉ sai bao nhiêu, mà còn sai theo kiểu nào?”**

21. Experiment 1 – Model Comparison

Câu hỏi: mô hình hồi quy nào cho kết quả tốt nhất dưới cùng train/test protocol?

Bảng results_h là bằng chứng định lượng. Không kết luận "Random Forest tốt nhất" hay "Gradient Boosting tốt nhất" nếu chưa nhìn vào metric thực tế.

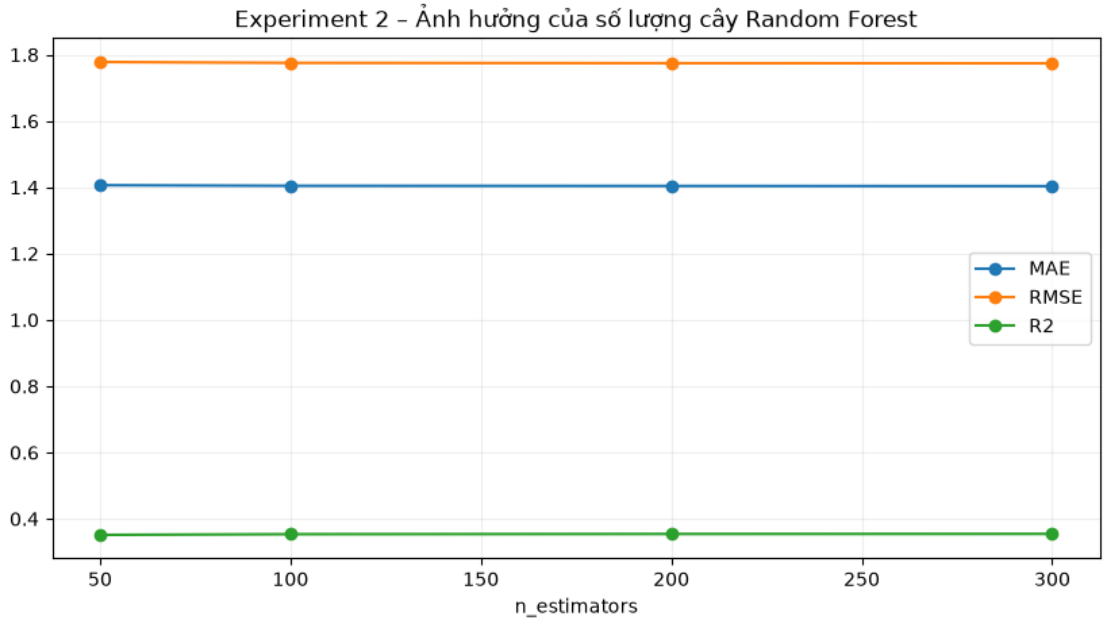
22. Experiment 2 – Hyperparameter Investigation

Câu hỏi: số lượng cây của Random Forest ảnh hưởng thế nào đến MAE/RMSE/R^2?

Chỉ thay đổi n_estimators, giữ các điều kiện khác cố định.

```
rf_exp=[]
for n in [50,100,200,300]:
    m=housing_pipeline(RandomForestRegressor(n_estimators=n,max_depth=20,n_jobs=-1,random_state=RANDOM_STATE))
    m.fit(X_train_h,y_train_h)
    p=m.predict(X_test_h)
    rf_exp.append({"n_estimators":n,
                   "MAE":mean_absolute_error(y_test_h,p),
                   "RMSE":mean_squared_error(y_test_h,p)**.5,
                   "R2":r2_score(y_test_h,p)})
rf_exp=pd.DataFrame(rf_exp)
display(rf_exp)
rf_exp.set_index("n_estimators")[["MAE","RMSE","R2"]].plot(marker="o",figsize=(10,5))
plt.title("Experiment 2 - Ảnh hưởng của số lượng cây Random Forest")
plt.grid(alpha=.2)
plt.show()
```

	n_estimators	MAE	RMSE	R2
0	50	1.406394	1.778076	0.351598
1	100	1.404553	1.775230	0.353671
2	200	1.404032	1.774322	0.354333
3	300	1.403542	1.773998	0.354569



23. Experiment 3 – Representation / Feature Investigation

Câu hỏi: categorical features có bổ sung thông tin dự đoán so với chỉ dùng numerical features hay không?

- Representation A: chỉ numerical features.
- Representation B: numerical + categorical features.

Cùng mô hình Ridge Regression và cùng train/test split.

```
# Representation A: numerical only
pre_num=Pipeline([("imputer",SimpleImputer(strategy="median")),("scaler",StandardScaler())])
m_num=Pipeline([("preprocessor",ColumnTransformer([("num",pre_num,num_h)],remainder="drop")),
                ("model",Ridge(alpha=10))])
m_num.fit(X_train_h,y_train_h)
p_num=m_num.predict(X_test_h)

# Representation B: all features
m_all=housing_pipeline(Ridge(alpha=10))
m_all.fit(X_train_h,y_train_h)
p_all=m_all.predict(X_test_h)

rep_h=pd.DataFrame([
    {"Representation":"Numerical only",
     "MAE":mean_absolute_error(y_test_h,p_num),
     "RMSE":mean_squared_error(y_test_h,p_num)**.5,
     "R2":r2_score(y_test_h,p_num)},
    {"Representation":"Numerical + Categorical",
     "MAE":mean_absolute_error(y_test_h,p_all),
     "RMSE":mean_squared_error(y_test_h,p_all)**.5,
     "R2":r2_score(y_test_h,p_all)}
])
display(rep_h)
```

	Representation	MAE	RMSE	R2
0	Numerical only	1.628989	1.989906	0.187901
1	Numerical + Categorical	1.424061	1.779902	0.350265

Kết luận Experiment 3

Nếu representation có thêm categorical features làm giảm MAE/RMSE hoặc tăng R^2, điều đó cho thấy các thuộc tính như hướng nhà, pháp lý, nội thất hoặc thông tin vị trí có thêm thông tin dự đoán ngoài các biến số. Nếu cải thiện nhỏ, điều đó cho thấy representation bổ sung chưa đem lại nhiều thông tin hữu ích cho mô hình đã chọn.

Kết luận cuối cùng phải dựa trên bảng rep_h thực tế, không suy diễn trước khi chạy.

24. Final Model – Giá nhà

```
best_h_name=results_h.iloc[0]["Model"]
final_housing_model=models_h[best_h_name]
final_housing_model.fit(X_train_h,y_train_h)

joblib.dump(final_housing_model,"artifacts/housing_final_model.joblib")
with open("artifacts/housing_model_info.json","w",encoding="utf-8") as f:
    json.dump({
        "model":best_h_name,
        "features":feature_cols_h,
        "target":"Price",
        "unit":"theo đơn vị của dataset"
    },f,ensure_ascii=False,indent=2)

print("Final housing model:",best_h_name)
print("Đã lưu artifacts/housing_final_model.joblib")
```

PHẦN IV – DEPLOY

25. Kiến trúc Web

Hai hệ thống được triển khai bằng Flask REST API:

Browser / Client

↓

HTML Form / JSON

↓

Flask API

↓

Same preprocessing + final model

↓

Prediction

↓

JSON / HTML result

Nguyên tắc: không viết lại preprocessing theo cách khác. Model đã lưu chứa preprocessing cần thiết trong Pipeline.

Link: https://assignment01-ai-app.onrender.com/

✨ Assignment 01 – Intelligent Systems

Hệ Thống Trí Tuệ Nhân Tạo & Dự Đoán

Ứng dụng Machine Learning hỗ trợ tầm soát bệnh học & định giá bất động sản

Dự Đoán Tiểu Đường | Định Giá Bất Động Sản

Tầm soát nguy cơ Bệnh Tiểu Đường (Diabetes Screening)

Mẫu: Bình thường | Mẫu: Nguy cơ cao

Mô hình phân loại: **Decision Tree Classifier** (Pima Indians Diabetes Dataset)

Số lần mang thai (Pregnancies)	Chỉ số Glucose (mg/dL)	Huyết áp tâm trương (mm Hg)	Độ dày nếp gấp da (mm)
6	148	72	35
Số lần từng mang thai	Bình thường: 70 – 140 mg/dL	Bình thường: 60 – 80 mm Hg	Vùng cơ tam đầu

Nồng độ Insulin (mu U/ml)	Chỉ số khối cơ thể (BMI)	Phả hệ tiểu đường (DPF)	Tuổi (Age)
0	33.6	0.627	50
2h sau khi nạp đường	kg/m² (Chuẩn: 18.5 – 24.9)	Chỉ số tiền sử gia đình	Độ tuổi hiện tại (năm)

Dự Đoán Nguy Cơ

⚠ Phát hiện nguy cơ mắc bệnh Tiểu đường

Xác suất dự báo có nguy cơ: **51.7%** (Nhãn phân loại: **1**)

* Lưu ý: Kết quả mang tính chất tham khảo giáo dục theo mô hình học máy, không thay thế chẩn đoán chuyên sâu từ bác sĩ chuyên khoa.

✨ Assignment 01 – Intelligent Systems

Hệ Thống Trí Tuệ Nhân Tạo & Dự Đoán

Ứng dụng Machine Learning hỗ trợ tầm soát bệnh học & định giá bất động sản

Dự Đoán Tiểu Đường | Định Giá Bất Động Sản

Định Giá Bất Động Sản Việt Nam (Housing Price Prediction)

Mô hình hồi quy: **Random Forest Regressor** (Dữ liệu thị trường Việt Nam)

Mẫu: Nhà Gò Vấp (TP.HCM) | Mẫu: Ocean Park (Hưng Yên/HN)

Địa chỉ / Dự án bất động sản

Dự án The Empire - Vinhomes Ocean Park 2, Xã Long Hưng, Văn Giang, Hưng Yên

Ví dụ: Tên đường, Phường, Quận/Huyện, Tỉnh/TP hoặc Tên dự án

Diện tích (m²)	Mặt tiền (m)	Độ rộng đường vào / ngõ (m)
91	7	13
	Độ rộng mặt tiền (để trống nếu không có)	Hẻm hoặc đường trước nhà

Số tầng	Số phòng ngủ	Số phòng tắm / vệ sinh
5	5	4

Hướng nhà	Hướng ban công	Tình trạng pháp lý
Tây - Bắc	Đông - Nam	Đã có sổ đỏ / Sổ hồng (Have certificate)

Tình trạng nội thất

Nội thất cơ bản (Basic)

Dự Đoán Giá Nhà

Kết Quả Định Giá Bất Động Sản

8,45 Tỷ VNĐ

Đơn giá ước tính: **~0.09 Tỷ VNĐ / m²** (92.8 Triệu/m²)

* Giá trị được ước lượng dựa trên mô hình Random Forest đối chiếu với các đặc trưng vị trí, diện tích, số phòng và pháp lý.

26. Kiến trúc Mobile

Mobile không huấn luyện lại mô hình. Mobile đóng vai trò giao diện nhập liệu và gọi REST API:

Android / Flutter

↓

JSON request

↓

Flask REST API

↓

Final model đã fit()

↓

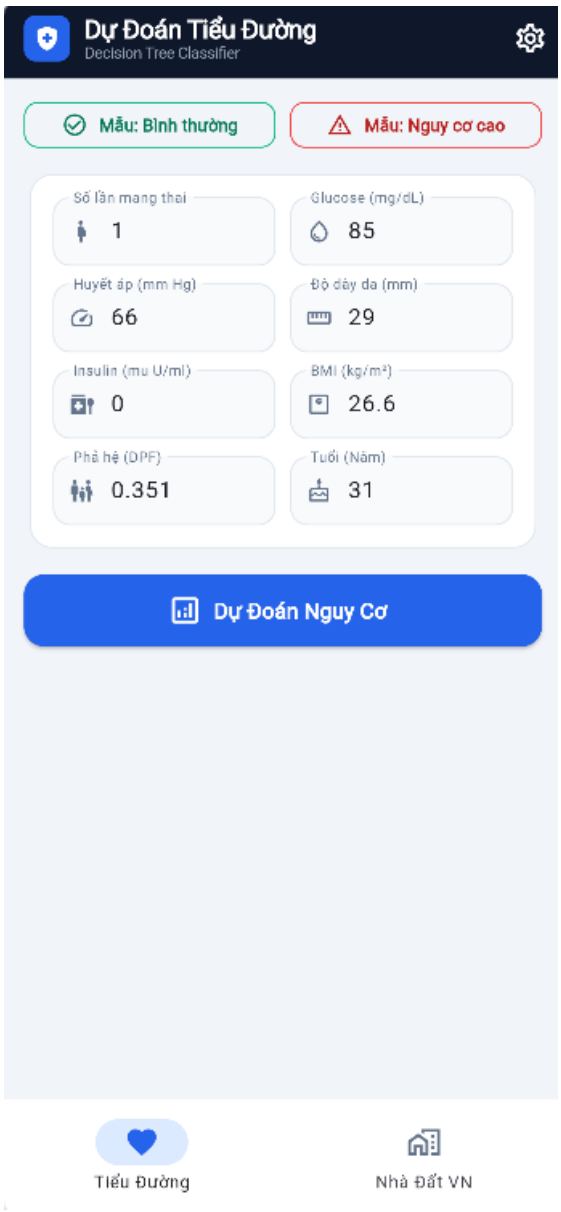
JSON response

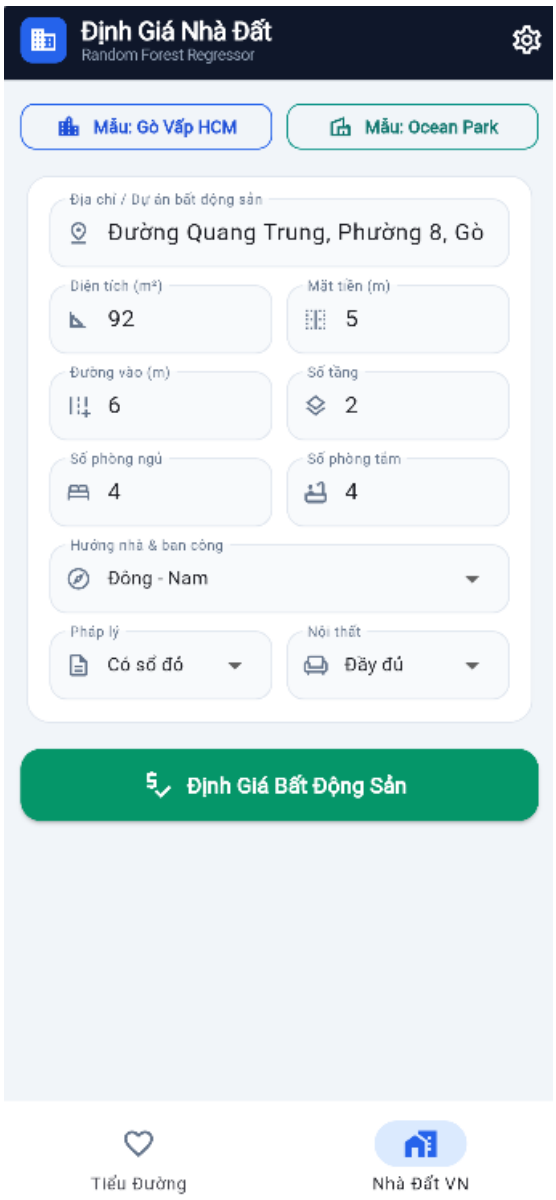
↓

Mobile hiển thị kết quả

Cách này giúp Web và Mobile sử dụng cùng một model, cùng preprocessing và cùng logic dự đoán.

Với Android Emulator, máy host thường được truy cập qua 10.0.2.2; với thiết bị thật cần dùng IP LAN của máy chạy Flask hoặc deploy API lên server.


Dự Đoán Tiểu Đường
Decision Tree Classifier
Mẫu: Bình thường
Mẫu: Nguy cơ cao
Số lần mang thai
1
Glucose (mg/dL)
85
Huyết áp (mm Hg)
66
Độ dày da (mm)
29
Insulin (mu U/ml)
0
BMI (kg/m²)
26.6
Phả hệ (DPF)
0.351
Tuổi (Năm)
31
Dự Đoán Nguy Cơ
Tiểu Đường
Nhà Đất VN


Định Giá Nhà Đất
Random Forest Regressor
Mẫu: Gò Vấp HCM
Mẫu: Ocean Park
Địa chỉ / Dự án bất động sản
Đường Quang Trung, Phường 8, Gò
Diện tích (m²)
92
Mặt tiền (m)
5
Đường vào (m)
6
Số tầng
2
Số phòng ngủ
4
Số phòng tắm
4
Hướng nhà & ban công
Đông - Nam
Pháp lý
Có sổ đỏ
Nội thất
Đầy đủ
Định Giá Bất Động Sản
Tiểu Đường
Nhà Đất VN